

Customer Relationship Management (CRM) System for Altrium

Module Code: COMP50074

Module name: PROFESSIONAL PRACTICE AND PROJECT
MANAGEMENT

Group Number: C 01

Thehan Andaramana: CB015685

Thumula Rumesh: CB015674

Sadev Ravishan: CB015733

Sivakumar Sabeeshan: CB011535

Acknowledgement

We would like to express our sincere gratitude to our lecturer, Dr. Tharanga Peiris, for her invaluable guidance, continuous feedback, and constructive critiques throughout this module.

We also extend our appreciation to Altrium for providing the business scenario that shaped the architecture and requirements of this CRM system.

Finally, we acknowledge the use of Anthropic's Claude as an AI tool for vibe coding, which assisted in rapid prototyping, generating boilerplate code, and streamlining our React frontend development.

Abstract

The Problem: With the current system at the Altrium, Sales teams often lose track of client information. Their information scattered across emails, spreadsheets, and people's memories. Without one central system, it is hard to tell which leads are active (hot) or inactive (cold), remember past conversations, or know when to follow up. Because of this disorganization, managers and company leaders cannot clearly see how the sales pipeline is doing. That makes it hard to make good business decisions.

The Proposed Solution: To fix this, we built a Customer Relationship Management (CRM) system. The system provides a central place for managing companies, contacts, leads, interactions, projects, phase tasks, approvals, and management information. A lead automatically has an associated project, which progresses through four controlled phases. Tasks, due dates, confirmations, and approval requests are used to control progress between phases. The system also implements role-based access for Sales Representatives, Sales Managers, Executive Managers, Project Managers, and System Administrators. Different roles receive different permissions according to their responsibilities.

Benefits to End Users: The CRM provides Sales Representatives with tools to manage assigned leads, record customer interactions, complete relevant tasks, and request approvals. Sales Managers can manage companies, leads, projects, requirement templates, approvals, and team-level pipeline information. Project Managers can manage the projects assigned to them and work on the relevant project tasks and execution activities. Executive Managers can perform management functions and decide Phase 4 sign-off requests. System Administrators can view system records and manage system settings and requirement templates while remaining restricted from normal record modification. The system also provides a dashboard, Kanban-style project board, calendar, management reporting, filtering, search, approval tracking, and a combined activity timeline.

Table of Contents

Acknowledgement.....	b
Abstract	c
Introduction.....	1
Sprint 1.....	3
1.1 Sprint Goal.....	3
1.2 Sprint Backlog.....	3
1.3 High Level Use Case Diagram (For the whole system)	4
1.4 Detailed Use Cases & User Stories	5
1.4.1 Use Case Diagram.....	5
1.4.2 User Stories	7
1.5 Activity Diagrams	9
1.6 Sequence Diagrams	12
1.7 Evidence of Testing	15
1.7.1 Manual Test Cases.....	15
1.8 Sprint Retrospective	18
1.8.1 What went well in the sprint	18
1.8.2 What went wrong in the sprint	18
1.8.3 What we commit to improve in the next sprint.....	18
1.8.4 Individual Retrospectives.....	18
Sprint 2.....	21
2.1 Sprint Goal.....	21
2.2 Sprint Backlog.....	22
2.3 Detailed Use Cases & User Stories	23
2.3.1 Use Case Diagram.....	23
2.3.2 User Stories	25
2.4 Activity Diagrams	28
2.5 Sequence Diagrams	32
2.6 Evidence of testing	35
2.6.1 Manual Test Cases.....	35
2.6.2 Automated Test Cases	38
2.7 Sprint Retrospective	40
2.7.1 What went well in the sprint	40
2.7.2 What went wrong in the sprint	40
2.7.3 What we commit to improve.....	40
2.7.4 Individual Retrospectives.....	40
Appendix.....	42
3.1 Project Management SCRUM board.....	42

3.1.1	Sprint 1	42
3.1.2	Sprint 2	43
3.2	Meeting Minutes of Daily Standup Meetings.....	44
3.3	Risk Register	45

Table of Figures

Figure 1: High Level Use Case Diagram (For the whole system)	4
Figure 2: Company & Contact Management Use Case Diagram	5
Figure 3: Lead Management Use Case Diagram	5
Figure 4: Interaction Logging Use Case Diagram	6
Figure 5: Search & Filter Use Case Diagram.....	6
Figure 6: Company & Contact Management Activity Diagram.....	9
Figure 7: Lead Management Activity Diagram for Add, Update, and View.....	9
Figure 8: Lead Management Activity Diagram for Archive	10
Figure 9: Interaction Logging Activity Diagram.....	10
Figure 10: Search & Filter Activity Diagram.....	11
Figure 11: Company & Contact Management Sequence Diagram	12
Figure 12: Lead Management Sequence Diagram for Add, Update, and View	12
Figure 13: Lead Management Sequence Diagram for Archive.....	13
Figure 14: Interaction Logging Sequence Diagram	13
Figure 15: Search & Filter Sequence Diagram	14
Figure 16: Dashboard & Reporting Process Use Case Diagram.....	23
Figure 17: Project Lifecycle & Phase Tasks Use Case Diagram.....	23
Figure 18: Lead Qualification & Status Management Use Case Diagram.....	24
Figure 19: Visual Pipeline & Phase Sign-off Use Case Diagram	24
Figure 20: Dashboard & Reporting Process Activity Diagram.....	28
Figure 21: Project Lifecycle & Phase Tasks Activity Diagram.....	29
Figure 22: Lead Qualification & Status Management Activity Diagram.....	30
Figure 23: Visual Pipeline & Phase Sign-off Activity Diagram.....	31
Figure 24: Dashboard & Reporting Process Sequence Diagram	32
Figure 25: Project Lifecycle & Phase Tasks Sequence Diagram	32
Figure 26: Lead Qualification & Status Management Sequence Diagram	33
Figure 27: Visual Pipeline & Phase Sign-off Sequence Diagram	34
Figure 28: Frontend Automated Testing Results	39
Figure 29: Backend Automated Testing Results	39
Figure 30: Sprint 1 Scrum Board	42
Figure 31: Sprint 2 Scrum Board	43
Figure 32 : 38th Meeting.....	44
Figure 33: 51st Meeting	44

List of Tables

Table 1: Sprint 1 Features with MosCoW Justifications	3
Table 2: Sprint 1 Manual Test Cases	17
Table 3: Sprint 1 Reflection Table	20
Table 4: Sprint 2 Features with MosCoW Justifications	22
Table 5: Sprint 2 Manual Test Cases	38
Table 6: Sprint 2 Reflection Table	41
Table 7 : Meeting Register	47

Introduction

Problem Statement

Altrium currently faces difficulty in managing customer and sales information because important details can be scattered across emails, spreadsheets, and individual records. This makes it difficult for sales staff and managers to track leads, review previous customer interactions, monitor project progress, and maintain a clear view of the sales pipeline. The proposed Customer Relationship Management (CRM) system provides a central platform for companies, contacts, leads, projects, tasks, approvals, and management information. The system uses role-based access so that users can perform functions appropriate to their responsibilities.

Alternative Solutions and Justification

Two alternative approaches were considered before developing the proposed solution. The first option was to continue using spreadsheets, email, and separate documents. This would require little initial development effort, but information would remain distributed across different locations and users, making tracking, permissions, approvals, and reporting more difficult.

The second option was to adopt an existing third-party CRM platform. This could provide many ready-made features and reduce development time. However, the project requires specific role-based permissions, controlled phase progression, task confirmation, approval workflows, and project-specific processes. A general CRM platform may require additional configuration to fit these requirements.

The selected option was therefore to develop a customised CRM prototype. This approach required more development effort but allowed the team to implement the required roles, workflows, project phases, approvals, and reporting features according to the scenario. The solution also provided a single integrated system for customer and project information.

Aim, Objectives and Deliverables

The aim of the project was to develop a CRM prototype that centralises customer, lead, interaction, and project information while supporting controlled role-based workflows.

The main objectives were to:

- provide central management of companies, contacts, leads, and interactions;
- implement role-based permissions for different CRM users;
- support controlled project phases, tasks, and approval requests;
- provide search, filtering, dashboards, reporting, and calendar functions; and
- validate the implemented functionality through manual and automated testing.

The main deliverables were the working CRM prototype, product and sprint backlogs, user stories, UML design diagrams, test evidence, Sprint 1 and Sprint 2 retrospectives, and project management documentation.

GitHub Repository

The complete source code and project files for the Altrium Customer Relationship Management (CRM) System are available in the group's GitHub repository.

<https://github.com/Thehan-Andaramana/Altrium-CRM>

The repository contains the implemented frontend, backend, database-related files, testing resources, and project documentation.

Sprint 1

1.1 Sprint Goal

The objective of Sprint 1 is to establish the core database foundation and functional user interface. It helps for managing company profiles, contacts, and the 3-phase lead pipeline. This sprint delivers critical record-keeping capabilities, interaction tracking, manager-confirmed phase tasks, and role-scoped permissions. By the end of this sprint, Sales Representatives and Managers will have a fully operational interface to capture leads and monitor daily sales engagement.

1.2 Sprint Backlog

No.	Feature Name	MoSCoW Prioritizing	Justification
1	Company & Contact Management	Must	View, create/update, assign/change company ownership, manage contacts, and controlled archiving of companies/contacts according to role.
2	Lead Management	Must	Create, view, update, assign, and manage leads according to role, including linking leads to companies and contacts.
3	Interaction & Logging	Must	Recording calls, emails, meetings, notes, and related activity provides a history of customer and project interactions.
4	Search & filter	Should	Search company and lead records, filter leads by status, phase, assigned Sales Representative, and Project Manager, and sort the displayed records
5	Lead Archive Workflow	Should	Allow authorized users to archive leads directly or request archiving according to their role and approval rules.
6	Activity Timeline	Could	Provide a combined lead timeline containing interactions and relevant system, approval, administrative, and phase events.
7	User Mentions & Task References	Could	Allow team members to mention users and reference project tasks from interaction notes, with notifications where the role/workflow requires them.

Table 1: Sprint 1 Features with MosCoW Justifications

1.3 High Level Use Case Diagram (For the whole system)

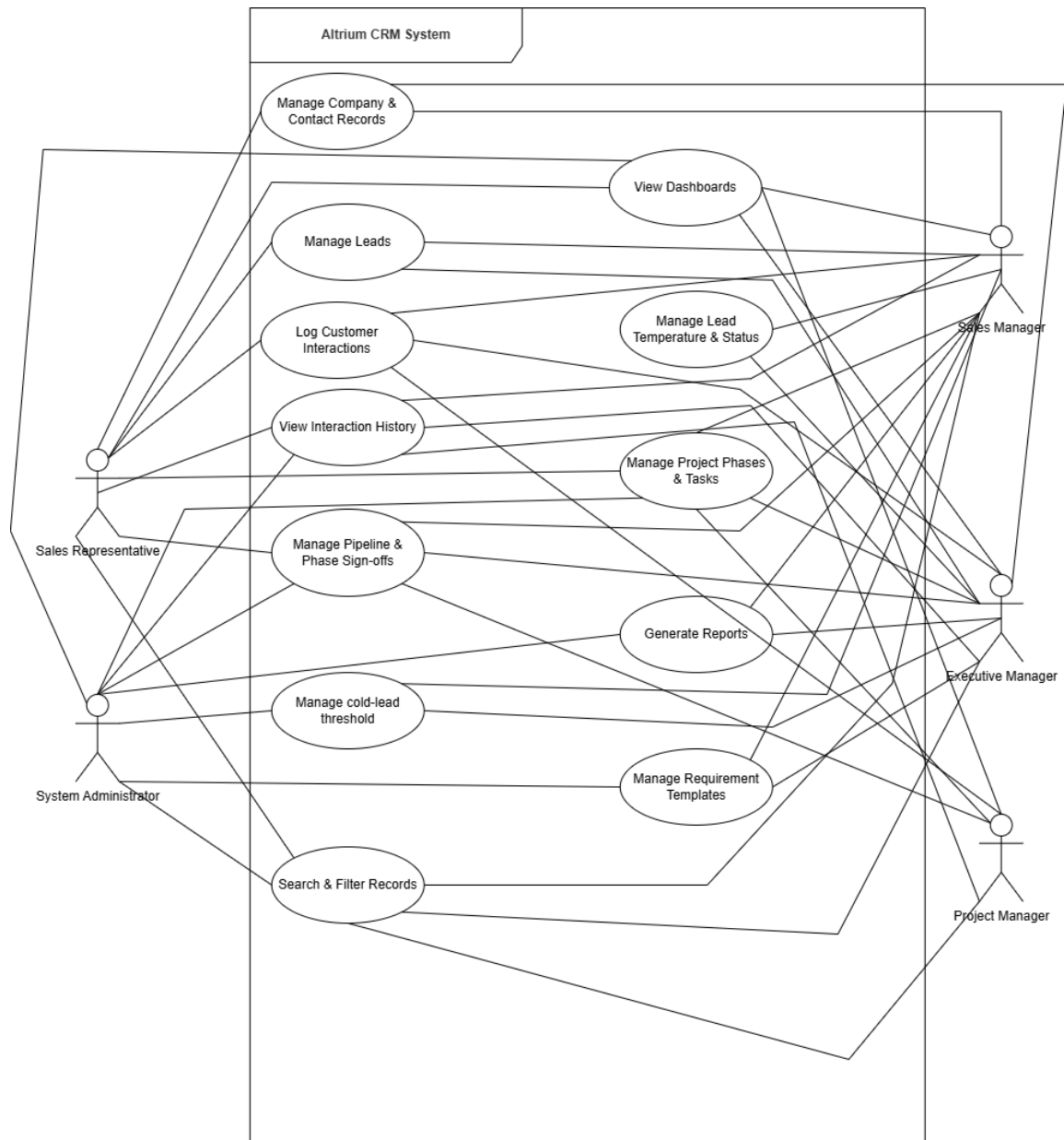


Figure 1: High Level Use Case Diagram (For the whole system)

1.4 Detailed Use Cases & User Stories

1.4.1 Use Case Diagram

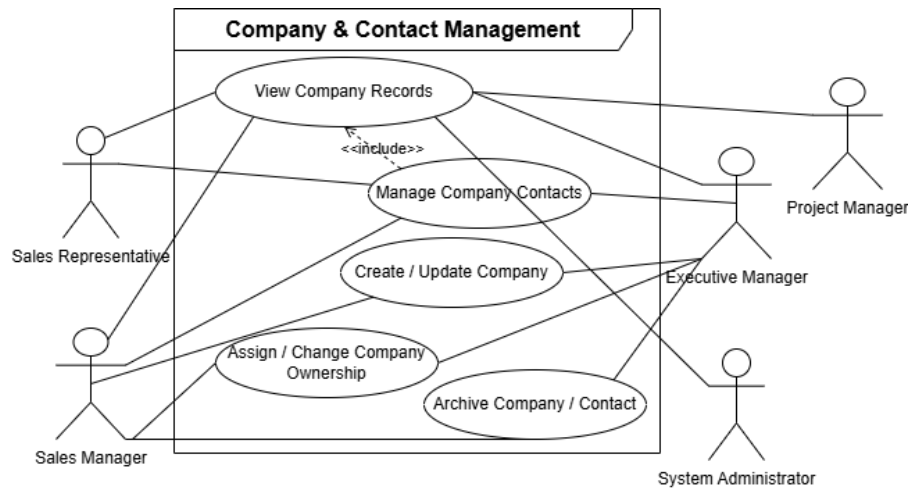


Figure 2: Company & Contact Management Use Case Diagram

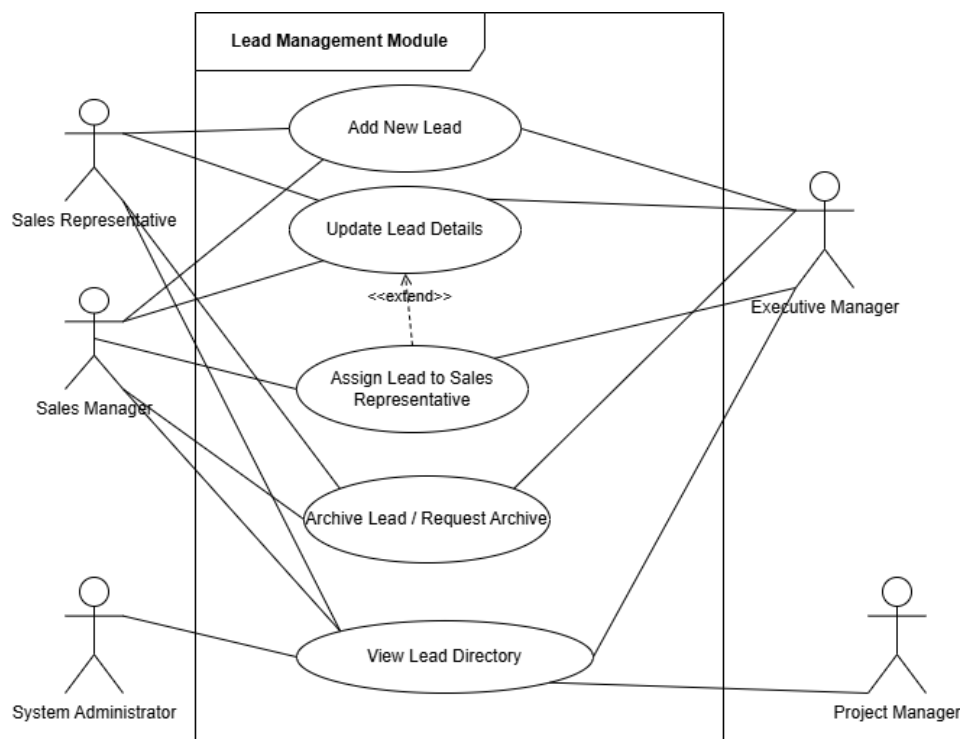


Figure 3: Lead Management Use Case Diagram

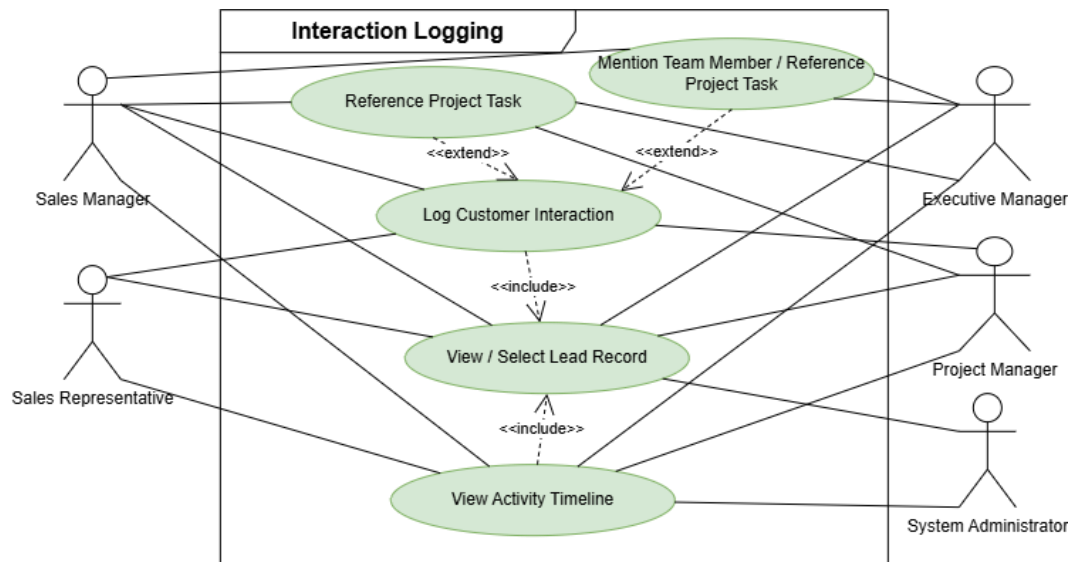


Figure 4: Interaction Logging Use Case Diagram

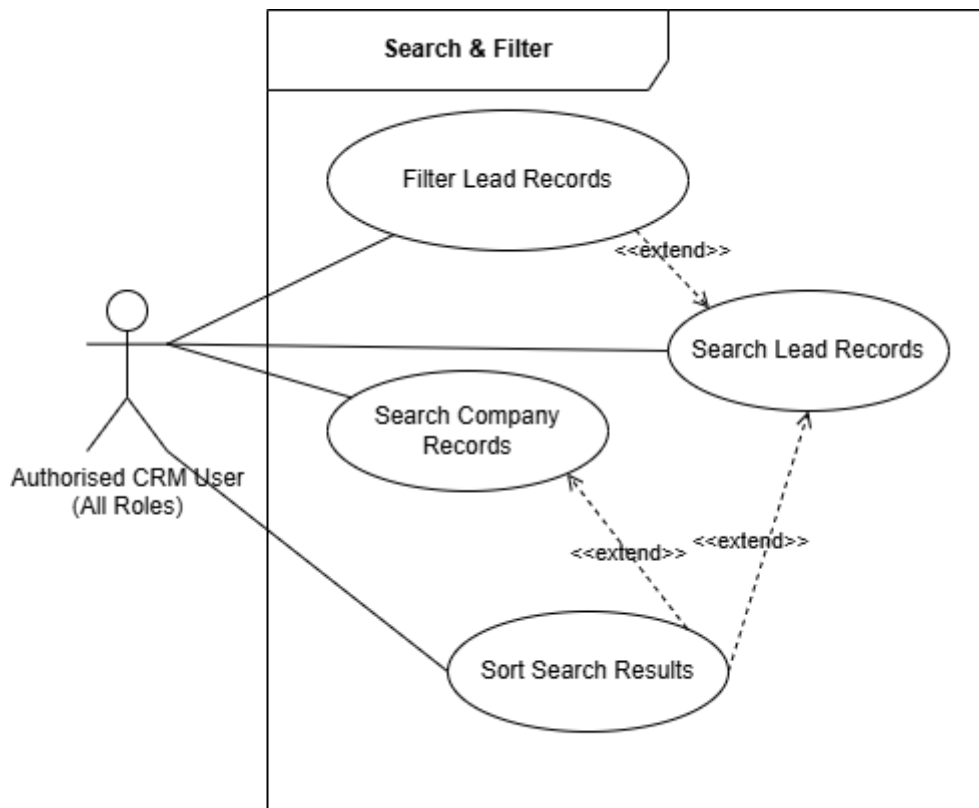


Figure 5: Search & Filter Use Case Diagram

1.4.2 User Stories

Company & Contact Management

US01:

As a CRM user, I need to view company records, so that I can find the organisation related to my work.

US02:

As a Sales Manager or Executive Manager, I need to create and update company records, so that the CRM contains accurate company information.

US03:

As a Sales Manager or Executive Manager, I need to assign or change company ownership, so that responsibility for company-related records is clear.

US04:

As a Sales Representative, Sales Manager, or Executive Manager, I need to add and update company contacts, so that customer contact information remains current.

Lead Management

US05:

As a CRM user, I need to view the lead directory according to my role, so that I can see the lead records relevant to my work.

US06:

As a Sales Representative, Sales Manager, or Executive Manager, I need to create a lead linked to a company and optionally to a contact, so that a new sales opportunity can enter the CRM.

US07:

As a Sales Manager or Executive Manager, I need to assign a lead to a Sales Representative, so that responsibility for the lead is clear.

US08:

As a Sales Representative, Sales Manager, or Executive Manager, I need to update lead information and perform or request archiving according to my role, so that pipeline information remains accurate while archive controls are maintained.

Interaction & Logging

US09:

As a CRM user, I need to view a lead's activity timeline, so that I can understand the history of customer and project-related activity.

US10:

As a Sales Representative, Sales Manager, Executive Manager, or Project Manager, I need to log calls, emails, meetings, and notes with outcomes, so that customer interactions are recorded in one place.

US11:

As a Sales Manager, Executive Manager, or Project Manager, I need to mention team members or reference project tasks in interaction notes, so that responsible users can be notified.

US12:

As a CRM user, I need to view interactions, approval events, administrative events, and phase events together, so that I can follow the complete history of a lead.

Search & Filter

US13:

As a CRM user, I need to search company records, so that I can quickly find the organisation I need.

US14:

As a CRM user, I need to search lead records, so that I can find a specific sales opportunity.

US15:

As a CRM user, I need to filter leads by status, phase, assigned Sales Representative, and Project Manager, so that I can focus on the records relevant to my work.

US16:

As a CRM user, I need to sort the displayed records, so that I can review them in the order I need.

1.5 Activity Diagrams

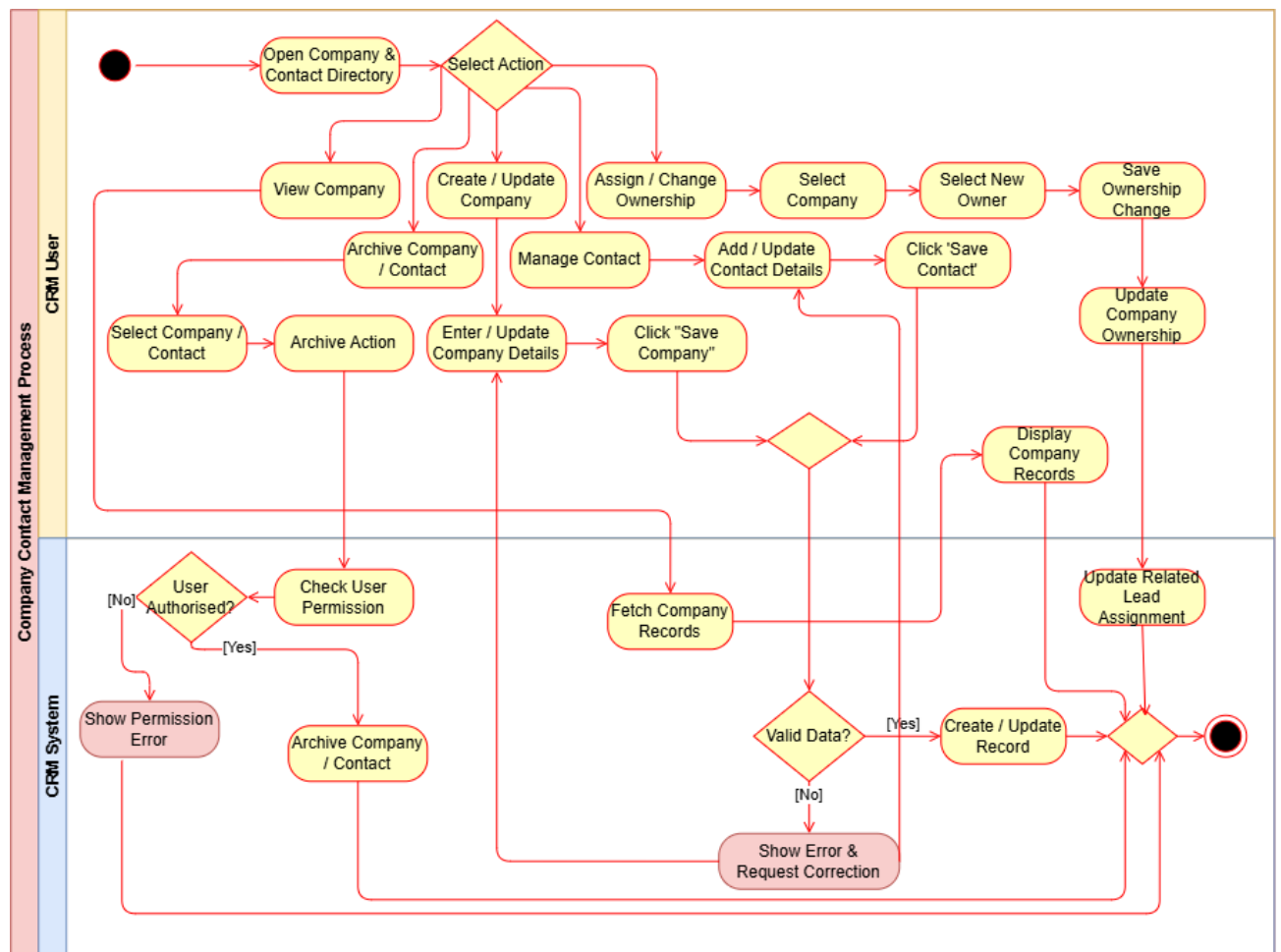


Figure 6: Company & Contact Management Activity Diagram

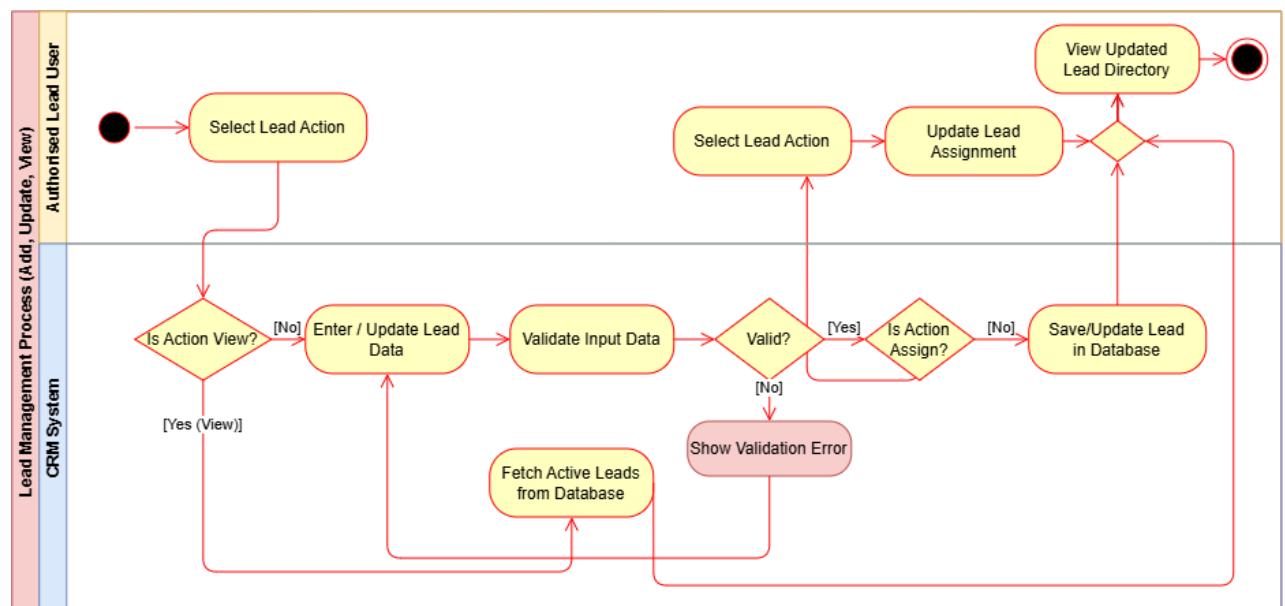


Figure 7: Lead Management Activity Diagram for Add, Update, and View

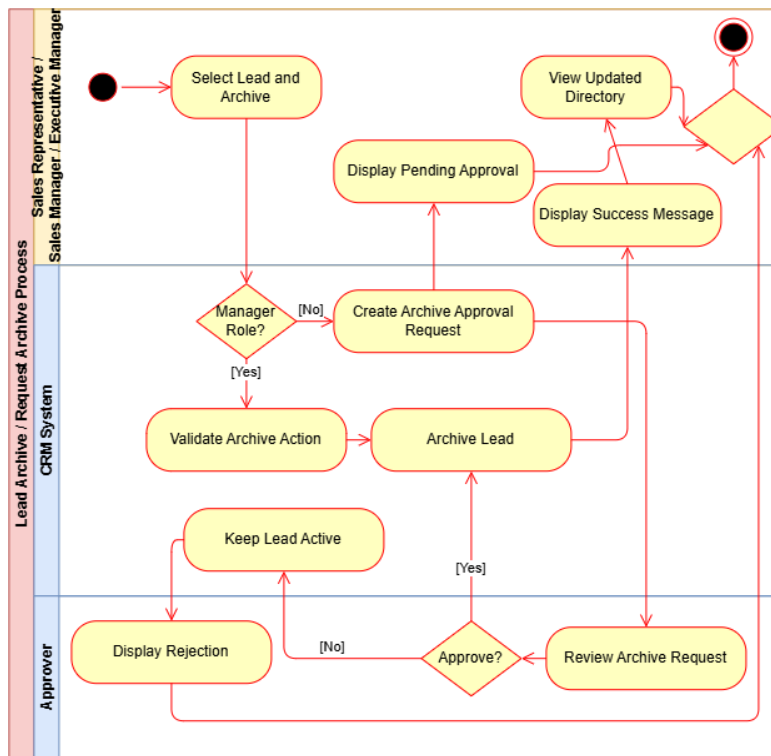


Figure 8: Lead Management Activity Diagram for Archive

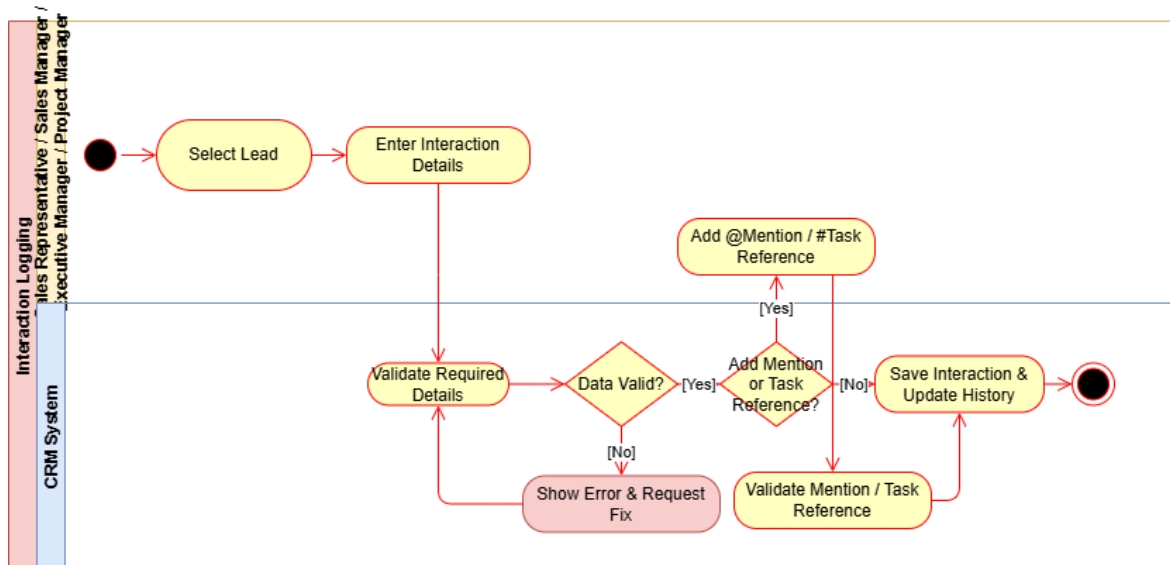


Figure 9: Interaction Logging Activity Diagram

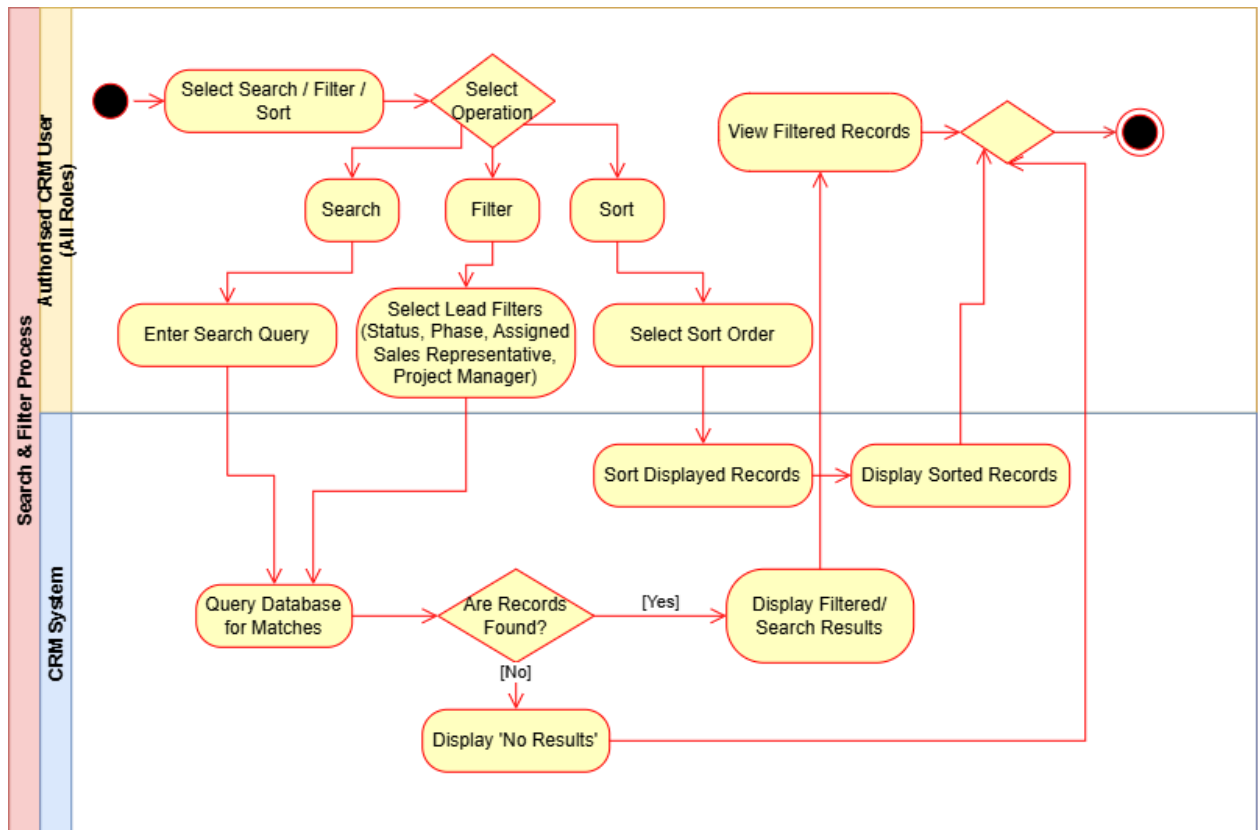


Figure 10: Search & Filter Activity Diagram

1.6 Sequence Diagrams

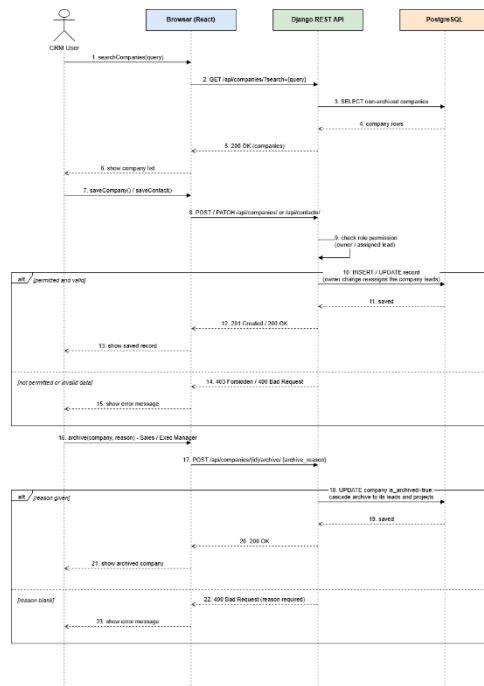


Figure 11: Company & Contact Management Sequence Diagram

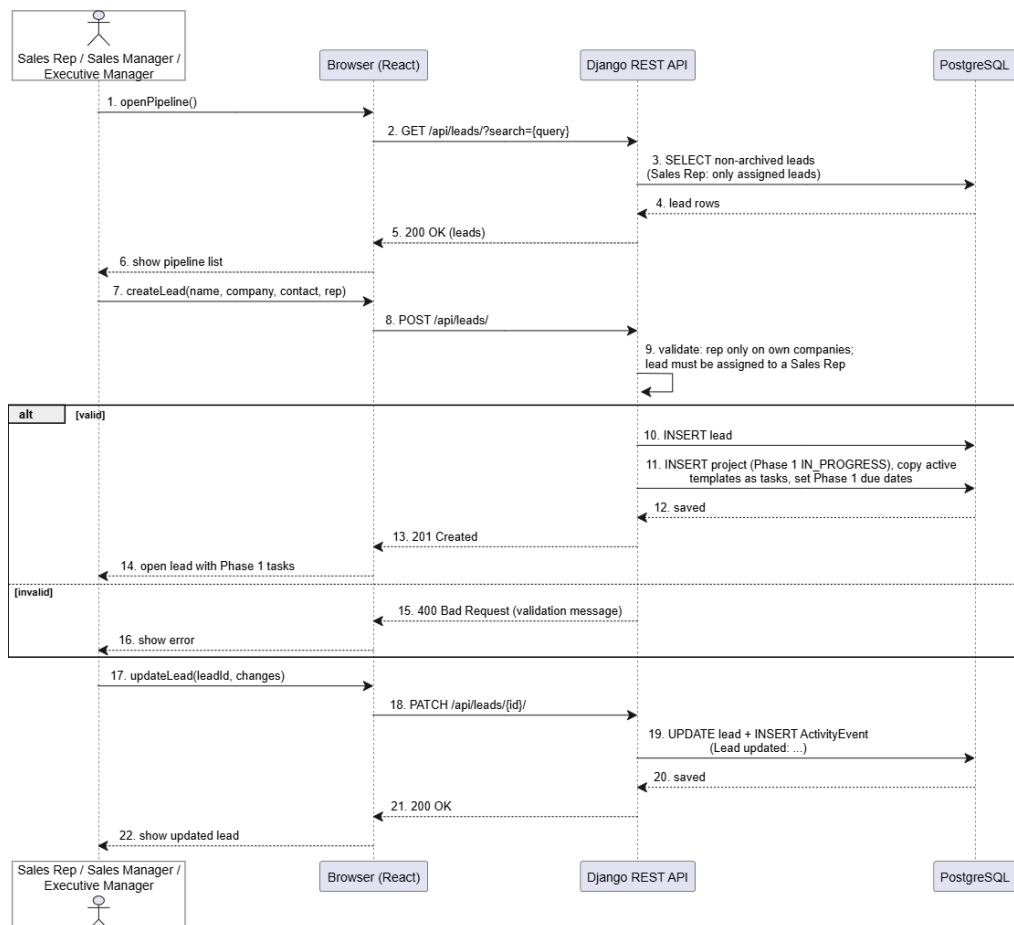


Figure 12: Lead Management Sequence Diagram for Add, Update, and View

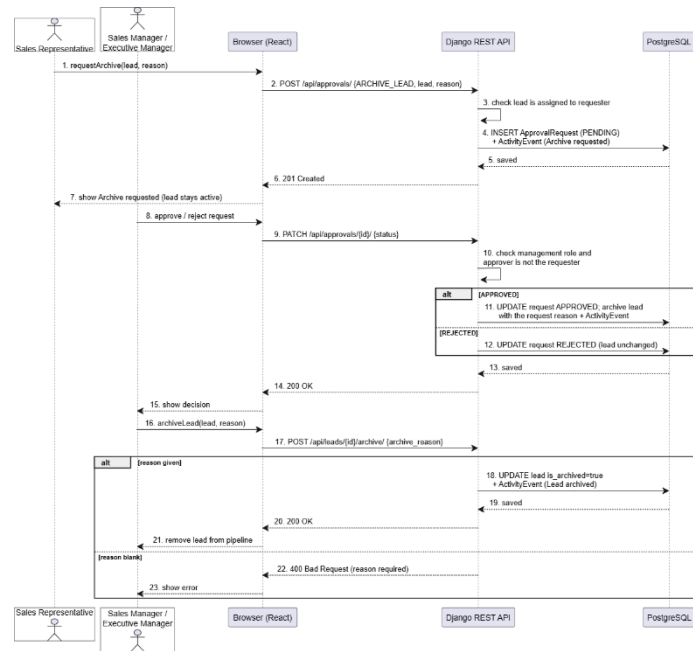


Figure 13: Lead Management Sequence Diagram for Archive

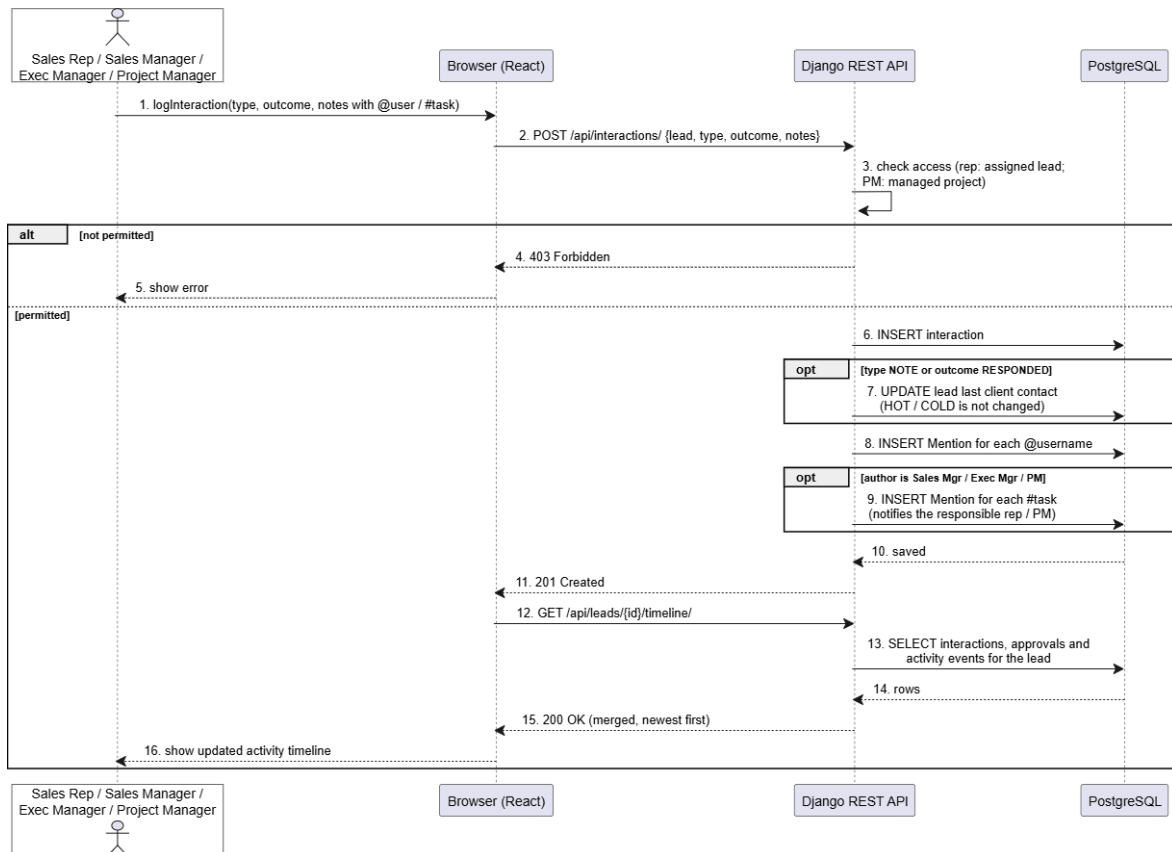


Figure 14: Interaction Logging Sequence Diagram

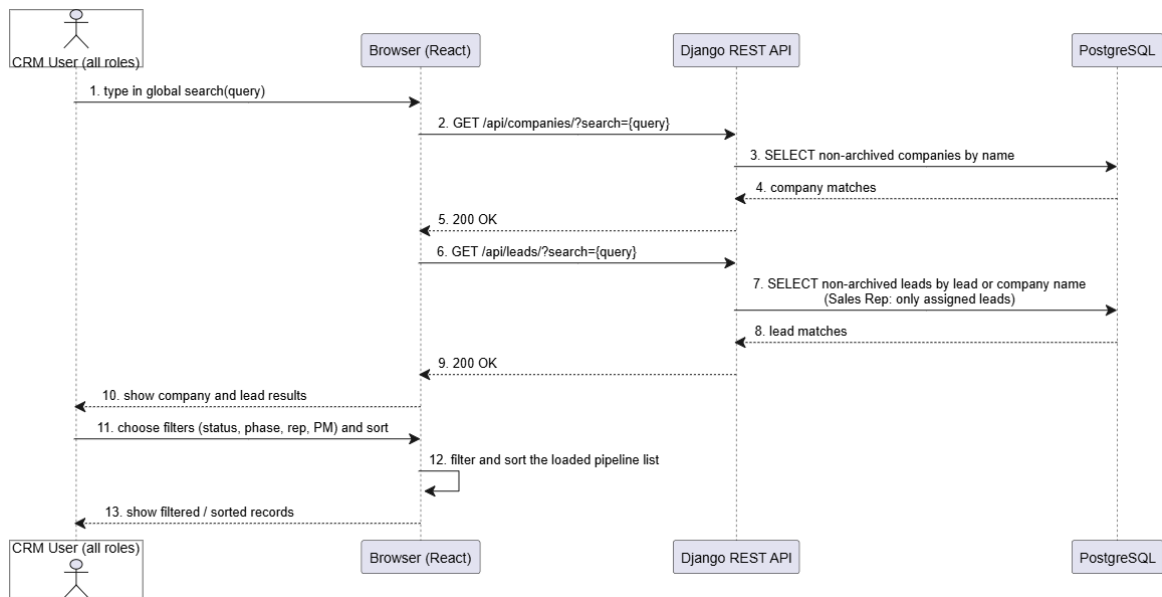


Figure 15: Search & Filter Sequence Diagram

1.7 Evidence of Testing

1.7.1 Manual Test Cases

Feature #	Feature name	Steps	Expected outcome	Actual outcome	Status
F00a	User Login (Sales Rep)	1. Go to the login page. 2. Log in as rep1.	The system logs the rep in and opens their dashboard.	Logged in as Sales Rep. The dashboard showed 2 hot and 1 cold lead, only rep1's own.	Pass
F00b	User Login (Sales Manager)	1. Go to the login page. 2. Log in as mgr1.	The system logs the manager in and opens the manager dashboard.	Logged in as Sales Manager. The dashboard showed all 4 hot and 4 cold leads, plus pending approvals and overdue tasks.	Pass
F00c	User Login (Executive Manager)	1. Go to the login page. 2. Log in as ex1.	The system logs the executive in with the same wide view as a manager.	Logged in as Executive Manager and saw the same full set of leads and approvals as the Sales Manager.	Pass
F00d	User Login (System Admin)	1. Go to the login page. 2. Log in as admin.	The system logs the admin in with a read-only view of records.	Logged in as System Admin. All records were visible, and Settings and Requirement Templates were reachable.	Pass
F00e	User Login (Project Manager)	1. Go to the login page. 2. Log in as pm1.	The Project Manager is logged in and sees only the projects they manage on the board and calendar.	Logged in as pm1. The board and calendar showed only the projects pm1 manages.	Pass
F00f	User Login (wrong password)	1. Go to the login page. 2. Log in as mgr1 with a made-up password.	Login is refused and an error is shown.	Refused with "Invalid credentials". We stayed on the login page.	Pass
F01a	Company Profile (Create)	1. Log in as mgr1 and open Companies. 2. Click New Company. 3. Fill in the details and save.	The company is saved and shows in the list.	The company saved and appeared at the top of the list, which is sorted newest first.	Pass
F01b	Company Profile (Edit)	1. Open the company just created. 2. Change the industry and save.	The company updates and shows the new details.	The change saved and the updated industry showed straight away.	Pass
F01c	Company Profile (Create as Admin)	1. Log in as admin. 2. Open Companies and look for New Company.	The admin cannot create companies.	The New Company button was hidden, and trying to add one anyway was refused.	Pass
F01d	Company Profile (Rep, not their own)	1. Log in as rep1. 2. Open a company belonging to another rep. 3. Try to edit it.	The rep can read it but not change it.	rep1 opened Initech, which belongs to someone else, but the edit was refused so it stays read-only.	Pass

F01e	Company Profile (Archive)	1. Log in as mgr1 and open a company with a lead. 2. Click Archive and give a reason.	The company and everything under it are archived together.	Acme Corp archived along with its lead and project, each showing the reason came from the company archive. Nothing was deleted.	Pass
F01f	Company Profile (Archive, no reason)	1. Click Archive on any company. 2. Leave the reason blank and save.	The archive is blocked until a reason is given.	Blocked with "A reason is required to archive." Spaces on their own were rejected too.	Pass
F01g	Company Owner (Change)	1. As mgr1, open a company with leads. 2. Change the owner to rep2 and save.	The owner changes, the company's leads move to rep2, and the change shows on each lead's timeline.	The owner changed to rep2, the company's leads moved to rep2, and each lead's timeline recorded the change.	Pass
F02a	Contact Management (Add)	1. Open a company profile. 2. Add a contact and save.	The contact is saved under the parent company.	The contact saved under the company and also appeared in the contact dropdown when creating a lead.	Pass
F02b	Contact Management (Rep, no lead)	1. Log in as rep1. 2. Open a company where rep1 has no lead. 3. Try to add a contact.	The rep cannot add the contact.	The contact could not be added; the request was refused for rep1.	Pass
F03a	Lead Creation	1. Open Pipeline and click New Lead. 2. Name it, pick the company and contact. 3. Assign it to rep1 and save.	The lead is created. Phase 1 starts with its 3 tasks and due dates. Phases 2, 3 and 4 are not started.	The lead saved and Phase 1 started with its 3 tasks and due dates. Phases 2, 3 and 4 stayed not started.	Pass
F03d	Lead Archive (Sales Rep)	1. Log in as rep1 and open one of their leads. 2. Click Request archive and give a reason.	The lead is not archived; a request goes to a manager.	The lead stayed in the Pipeline and a request was raised for a manager. The rep could not archive it directly.	Pass
F03e	Lead Archive (Approve request)	1. Log in as mgr1 and open Approvals. 2. Approve rep1's archive request.	The lead is archived with the rep's reason and disappears from the Pipeline.	The lead was archived with rep1's reason and no longer appeared in the Pipeline.	Pass
F03f	Lead Archive (Manager, direct)	1. As mgr1, open a lead. 2. Click Archive and give a reason.	The lead is archived straight away and the archive shows on its timeline.	The lead was archived immediately and the archive appeared on its timeline.	Pass
F03g	Lead Assignment (Manager as owner)	1. As mgr1, create a lead. 2. Try to assign it to mgr1 instead of a rep.	Refused: a lead can only be assigned to a sales rep.	Refused with "A lead can only be assigned to a sales rep."	Pass

F06a	Interaction (Client answered)	1. Open a Cold lead. 2. Log a call with the outcome Responded.	"Last client contact" updates to today and the call appears on the timeline. The lead stays Cold.	"Last client contact" updated to today and the call appeared on the timeline. The lead stayed Cold.	Pass
F06b	Interaction (No answer)	1. Set the lead back to Cold. 2. Log a call with the outcome No Answer.	The lead stays Cold.	The lead stayed Cold and "last client contact" did not move. Only the timeline recorded the attempt, so chasing a quiet client does not make it look active.	Pass
F06d	Interaction (Note)	1. Open a lead. 2. Log a Note with some text.	The note saves without an outcome and appears at the top of the timeline.	The note saved with no outcome and appeared at the top of the timeline.	Pass
F06e	Interaction (Mention)	1. As mgr1, log a note containing @rep1. 2. Log in as rep1.	rep1 has an unread notification linking to the lead.	rep1 had an unread notification that linked to the lead.	Pass
F06f	Activity Timeline	1. Open a lead that has interactions, an approval and a task change. 2. Open the Activity tab.	Interactions, approvals and system events all appear in one list, newest first.	Interactions, approvals and system events appeared together in one list, newest first.	Pass
F07a	Search (Companies)	1. Log in as rep1. 2. Search for another rep's company by name.	The company is found even though the rep does not own it.	rep1 found Initech, which belongs to someone else, and could open it read-only.	Pass
F07b	Filter (Leads by status)	1. Open Pipeline. 2. Filter by Hot, then Cold, then clear it.	The list narrows correctly and the full list comes back.	4 Hot and 4 Cold, which adds up to the full list of 8. Clearing the filter brought everything back.	Pass
F07c	Search (Leads as Rep)	1. Log in as rep1. 2. Search for the name of a lead assigned to rep2.	No result is shown; reps only find their own leads.	No result was shown for rep2's lead.	Pass
F07d	Sort and filter (Pipeline)	1. Open Pipeline. 2. Filter by phase and by project manager. 3. Change the sort chip.	The list narrows and reorders correctly on screen.	The list narrowed by phase and project manager, and the order changed with the sort chip.	Pass

Table 2: Sprint 1 Manual Test Cases

1.8 Sprint Retrospective

1.8.1 What went well in the sprint

We used our technical skills to quickly build the React code and update our pipeline design. It showed strong "Implementer" and "Specialist" teamwork styles. We also did a great job managing our own work based on the lecturer's feedback. Everyone took responsibility and did their work without needing to be micromanaged, which is a great example of McGregor's "Theory Y" behavior.

1.8.2 What went wrong in the sprint

The team fractured into isolated silos, plunging us into a chaotic "Storming" stage (Tuckman). The developers coded the actual system while the Business Analysts documented an entirely different, idealized version of the website. Because nobody acted as a proper "Coordinator" to connect the coders with the writers. Our original plans were completely different from what actually coded. This caused a huge, stressful panic where we had to rewrite almost all our paperwork to match the real code.

1.8.3 What we commit to improve in the next sprint

To stop the chaos and start working as a real team ("Norming" stage), we must stop working in isolation. We promise that developers and analysts will sit down together at Sprint Planning so our user stories and UML diagrams only include features the developers can actually build. We will also enforce a strict rule. No task is called "Done" until the code works and the documents match the real system.

1.8.4 Individual Retrospectives

Member	Roles	Experience	Reflection
Thehan (CB015685)	Scrum Master	• Managed development throughput. • Facilitated team unblocking.	As Scrum Master, I was responsible for facilitating the team and removing blockers. I neglected my Coordinator responsibilities (Belbin) to maintain process discipline. By allowing the Agile board to fall out of sync with actual progress, I failed to facilitate transparency, leading to integration issues indicative of the Storming phase (Tuckman). Next sprint, I will adopt a more Theory Y approach (McGregor), empowering the team to track their own work while dedicating specific time to verify frontend-backend integration before allowing tickets to be marked "Done".
	Developer	• Built core React UI components.	As a Developer, I contributed technically by rapidly building the React components.

		Implemented frontend logic for the pipeline.	I exhibited strong Implementer (Belbin) traits by focusing heavily on pushing code before the deadline. However, I worked in isolation and did not collaborate effectively with the backend developer, which caused integration bottlenecks. I learned that speed without alignment causes delays. Next sprint, I will balance my coding focus with stronger Teamworker (Belbin) communication to ensure my components integrate smoothly.
Thumula (CB015674)	Business Analyst	- Drafted initial project plan and requirements. - Created UML diagrams and Use Cases.	I drafted the initial project plan, user stories, and UML diagrams for the team. The truth is I operated in a complete silo as a Plant (Belbin), designing an idealized system (with features like e-signatures and analytics) while entirely disconnected from what the developers were actually capable of building within the timeframe. My documentation was so out of sync with the codebase that I had to frantically reverse-engineer and rewrite the entire backlog and UML on submission day to match the developer's reality. Next sprint, I will stop designing in a vacuum. I need to collaborate directly with the developers from day one so my requirements match technical reality.
	Scrum Master	- Managed the GitHub project board. - Updated tickets with the correct User Stories. - Managed sprint scope and backlog alignment.	I was supposed to keep the team's scope aligned and manage the board. In reality, I was an absent Coordinator (Belbin). I didn't verify the actual code against our tickets, leading to jam at work. We were deep in a chaotic Storming phase (Tuckman) because the developer was building one thing while I was documenting another, forcing an 11th-hour scramble to strip out unbuilt features. A Scrum Master cannot just manage theoretical tickets. Next time, I will adopt an active Theory Y approach (McGregor). Regularly reviewing the actual working software with the team to ensure our tracking matches reality.
Sabeeshan (CB011535)	Developer	At the beginning, I struggled to understand the conceptualization.	Phase 1 was challenging because I was new to both the project and Agile methodology. Initially, I struggled with conceptualization, requirements, and daily meetings. However, as

			the phase progressed, I became more comfortable with the process. The MoSCoW and UI/UX workshops helped me understand the project and its requirements more clearly. Overall, Phase 1 improved my understanding of Agile teamwork, requirements, conceptualization, and UI/UX design. It also helped me become more confident in participating in an Agile team.
	Business Analyst	• complete the pending work from the previous BA • created the use case, activity, and sequence diagrams for all the remaining functional features	Phase 2 was a new experience for me because I moved from the Developer role to the Business Analyst role. At first, I had to understand the work already completed by the previous BA before continuing with the remaining tasks. The existing diagrams helped me understand the expected format and system flow. I learned the importance of maintaining consistency when different team members work on the same project. This phase also improved my understanding of functional requirements and gave me more confidence in creating use case, activity, and sequence diagrams.
Sadev (CB015733)	Quality Analyst	• Oversaw manual testing. • Verified component functionality.	I was tasked with integrating quality into the sprint by testing the final outputs. I operated strictly as a Completer Finisher (Belbin). Because I waited until the very end of the workflow, and because tickets were prematurely marked "Done", my testing was severely delayed and rushed. I learned that quality cannot just be checked at the end. Next sprint, I will participate in Sprint Planning to define clear "Definition of Done" criteria and write test cases concurrently with development.

Table 3: Sprint 1 Reflection Table

Sprint 2

2.1 Sprint Goal

The objective of Sprint 2 is to extend the core CRM into a controlled project and sales pipeline management system.

This sprint introduces controlled HOT/COLD lead status management, the visual project pipeline, the four-phase project lifecycle, phase tasks, requirement templates, due dates, approvals, dashboards, management reporting, system settings, task scheduling, and supporting collaboration and personalisation features.

By the end of Sprint 2, the CRM supports the controlled workflow from lead management through project phases, task completion, approval, monitoring, and management reporting.

The project lifecycle contains four phases:

Phase 1 – Initial/Pre-Sale Activities

- Client Proposal Confirmation
- Requirement Discussion
- Contract Papers

Phase 2 – Analysis and Planning

- Detailed Requirements Analysis
- Budget Proposal
- Technical Analysis
- Feasibility Study
- Project Planning

Phase 3 – Technical Delivery

- Technical Specification
- Development Progress Review
- QA Sign-off

Phase 4 – Completion and Handover

- Client Acceptance
- Final Proposal Signature
- Handover Note

Progress between phases is controlled through task completion and approval requests.

2.2 Sprint Backlog

No.	Feature Name	MoSCoW Prioritizing	Justification
1	Lead Qualification & Status Management	Must	Lead status is a core CRM function and requires controlled HOT/COLD changes, role-based permissions, reasons, and approval tracking.
2	Visual Pipeline & Phase Sign-offs	Must	The project requires a visual view of project progress and controlled approval before a project can move to the next phase.
3	Project Lifecycle & Phase Tasks	Must	Projects require four controlled phases, assigned tasks, due dates, task forms, attachments, task confirmation, and role-based task completion.
4	Dashboards & Operational Alerts	Must	Users need immediate visibility of HOT/COLD leads, pending approvals, overdue tasks, and active projects to monitor operational work.
5	Management Reporting	Should	Management reports provide useful analysis of projects, phases, users, approvals, budgets, and interactions, but the core CRM workflow can operate without them.
6	Requirement Templates	Should	Editable requirement templates improve consistency and allow managers to configure standard project tasks without changing application code.
7	System Settings	Should	The configurable cold-lead threshold allows management to control an important operational rule used by the CRM.
8	Task Calendar	Should	The calendar provides an additional view of task due dates and helps users identify upcoming, due-soon, overdue, and completed work.
9	Personal Preferences	Could	Theme, font size, density, card style, sidebar, and notification settings improve usability but are not essential to the core business workflow.
10	Mentions & Notifications	Could	User mentions and mention notifications improve team communication and awareness but are supporting collaboration features rather than core CRM operations.

Table 4: Sprint 2 Features with MosCoW Justifications

2.3 Detailed Use Cases & User Stories

2.3.1 Use Case Diagram

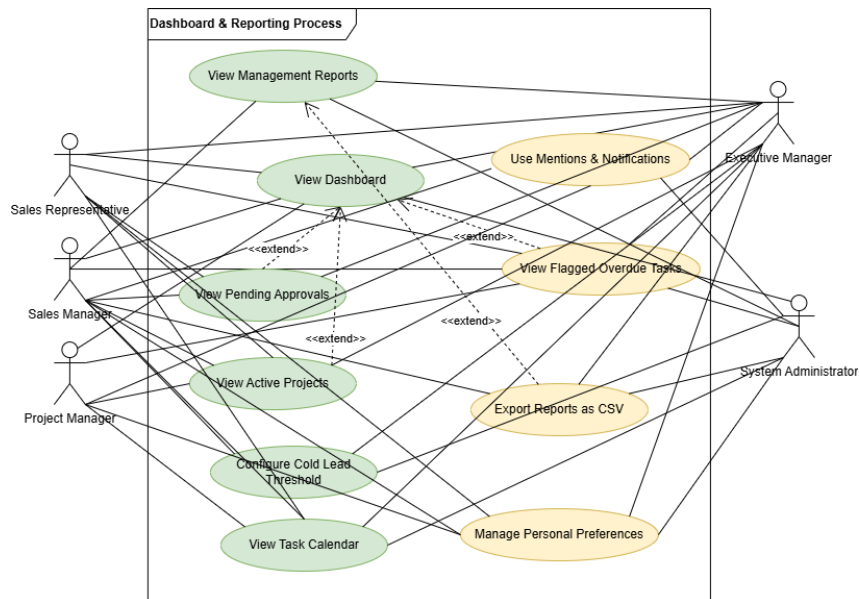


Figure 16: Dashboard & Reporting Process Use Case Diagram

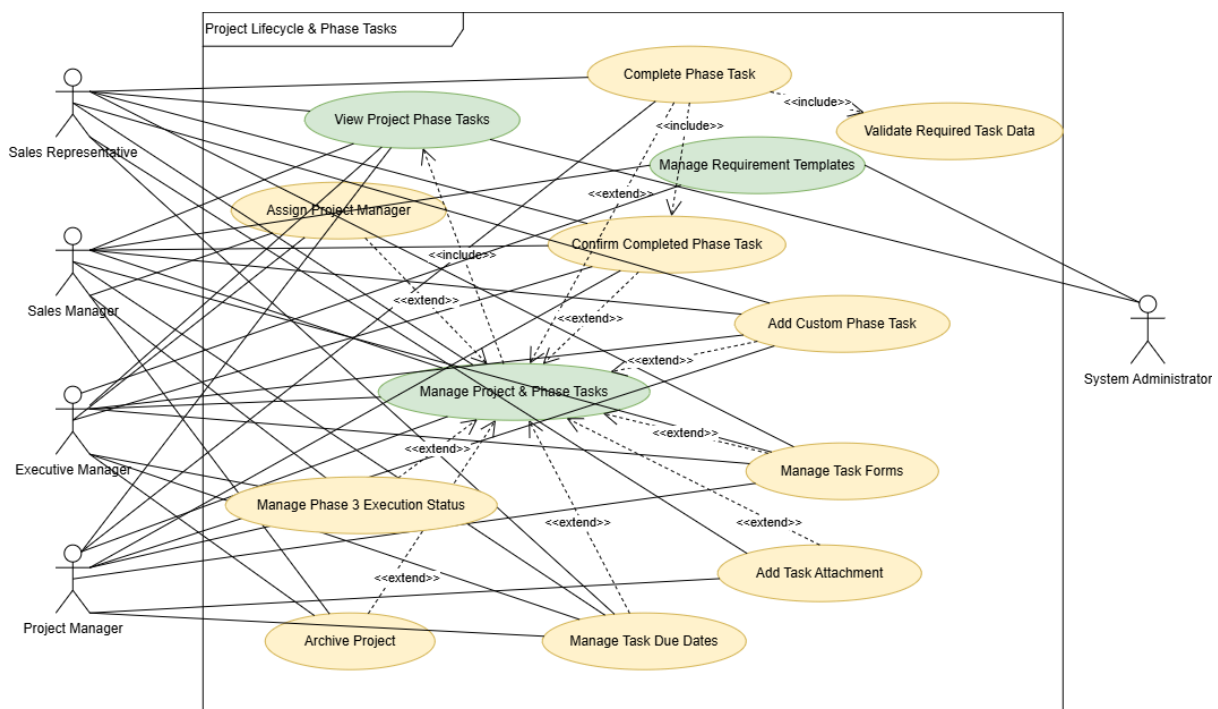


Figure 17: Project Lifecycle & Phase Tasks Use Case Diagram

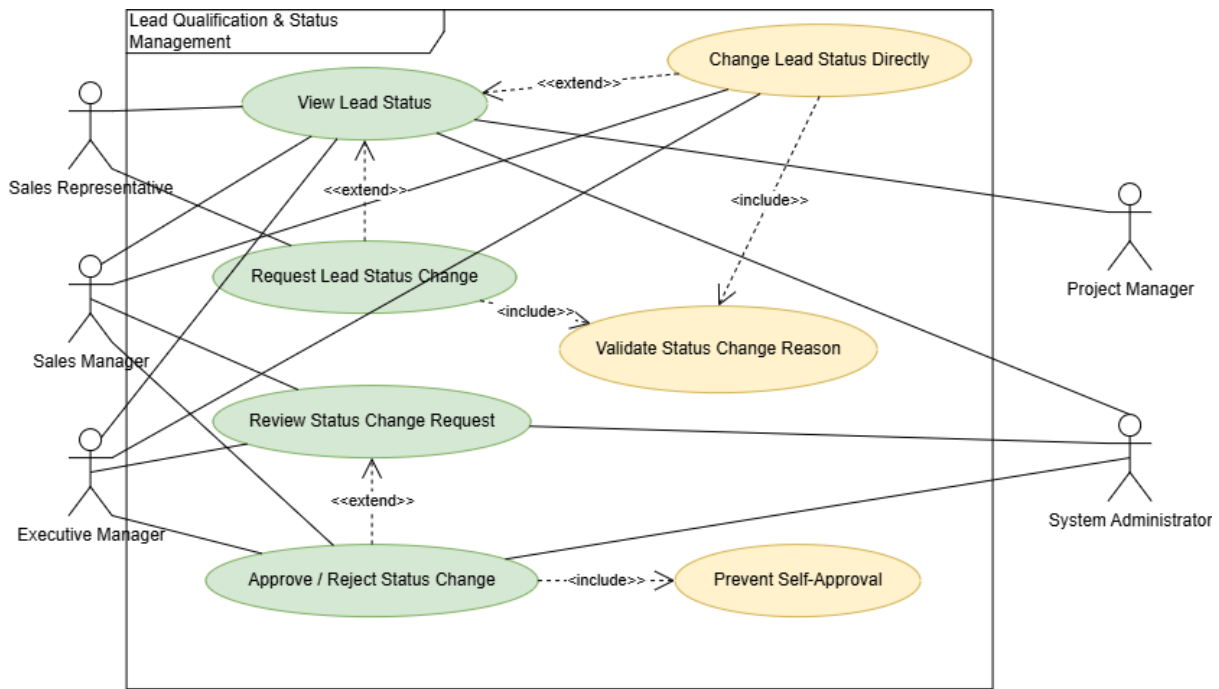


Figure 18: Lead Qualification & Status Management Use Case Diagram

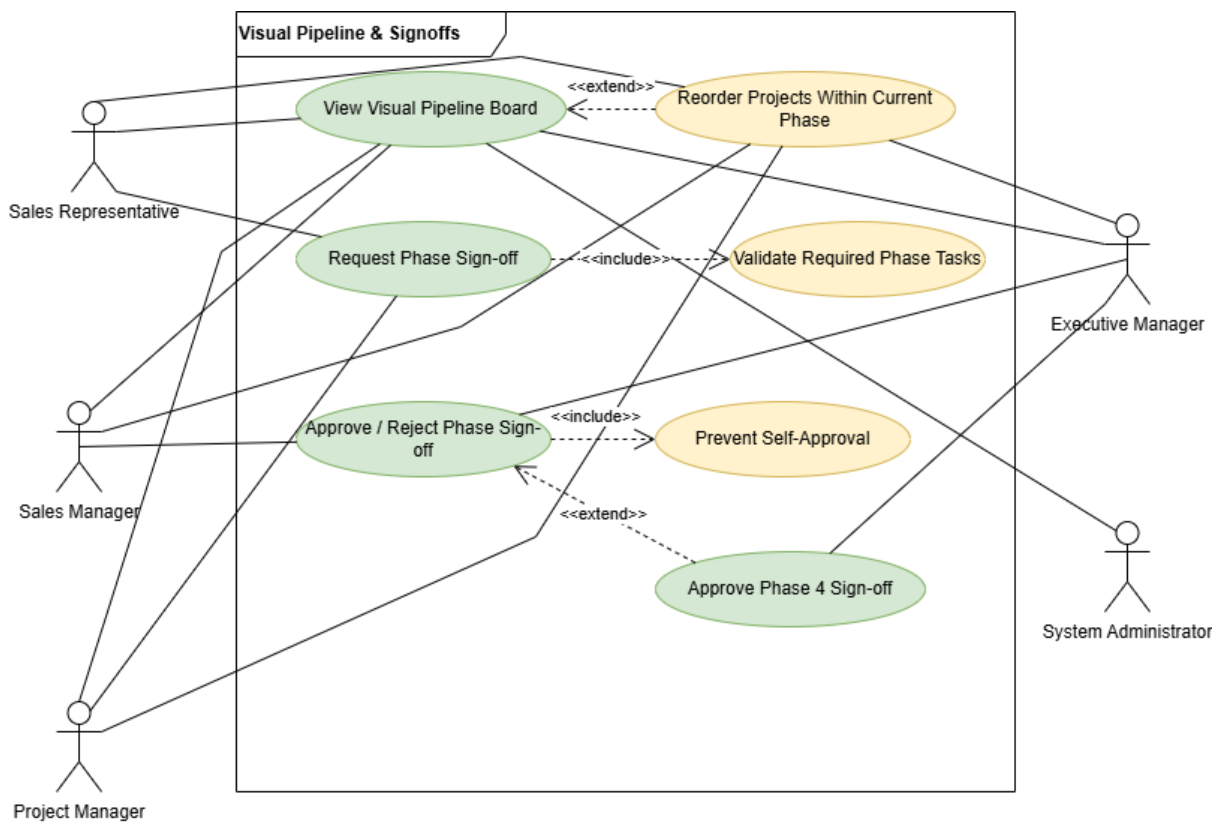


Figure 19: Visual Pipeline & Phase Sign-off Use Case Diagram

2.3.2 User Stories

Lead Qualification & Status Management

US17:

As a CRM user, I need to view whether a lead is HOT or COLD, so that I can understand the current lead temperature.

US18:

As a Sales Manager or Executive Manager, I need to change a lead between HOT and COLD with a reason, so that important status changes are properly recorded.

US19:

As a Sales Representative, I need to request a HOT or COLD status change with a reason, so that I can submit changes while following the approval process.

US20:

As a Sales Manager, Executive Manager, or System Administrator, I need to approve or reject lead status change requests, so that lead qualification changes are controlled.

Visual Pipeline & Phase Sign-offs

US21:

As a CRM user, I need to view project cards grouped by their current phase, so that I can understand the project pipeline visually.

US22:

As a Sales Representative, Sales Manager, Executive Manager, or Project Manager, I need to reorder project cards within the current phase, so that the board reflects the working order without changing the project phase.

US23:

As an authorised project user, I need to request a phase sign-off after the required work is completed, so that a project can move forward through the controlled lifecycle.

US24:

As an authorised management user, I need to approve or reject phase sign-off requests, so that projects advance only after the required approval. Phase 4 sign-off is decided by the Executive Manager.

Project Lifecycle & Phase Tasks

US25:

As a Sales Manager or Executive Manager, I need to assign a Project Manager to a project, so that project responsibilities are clearly assigned.

US26:

As a responsible project user, I need to complete phase tasks, enter task notes, answer required task forms, and add supporting attachments, so that project work is properly recorded.

US27:

As an authorised project user, I need to manage project task information such as due dates and additional tasks, so that project activities can be organised according to project requirements.

US28:

As a Project Manager or authorised management user, I need to manage Phase 3 execution status, so that technical delivery progress is controlled before the required approval.

US29:

As a Sales Manager or Executive Manager, I need to archive projects according to the project lifecycle rules, so that completed or inactive projects can be controlled without removing their records.

Requirement Templates

US30:

As a Sales Manager, Executive Manager, or System Administrator, I need to create, edit, reorder, copy, activate, and deactivate requirement templates, so that projects can use a consistent task structure.

Task Calendar

US31:

As a CRM user, I need to view relevant task due dates and statuses in a calendar, so that upcoming, due-soon, overdue, and completed work can be tracked.

Dashboards & Operational Alerts

US32:

As a CRM user, I need to view a role-based dashboard containing relevant leads, projects, approvals, and overdue work, so that I can identify the work that requires attention.

Management Reporting

US33:

As a Sales Manager, Executive Manager, or System Administrator, I need to generate management reports for a selected period, so that project, approval, and interaction activity can be reviewed.

US34:

As a Sales Manager, Executive Manager, or System Administrator, I need to export report data, so that reporting information can be used outside the CRM.

System Settings

US35:

As a Sales Manager, Executive Manager, or System Administrator, I need to configure the cold-lead threshold, so that the system can identify approaching cold leads according to the organisation's setting.

Personal Preferences

US36:

As a CRM user, I need to manage my personal display and notification preferences, so that the CRM interface can be configured for my working needs.

Mentions & Notifications

US37:

As a CRM user, I need to mention team members and reference project tasks in interactions, so that responsible users can receive relevant notifications and follow-up information.

2.4 Activity Diagrams

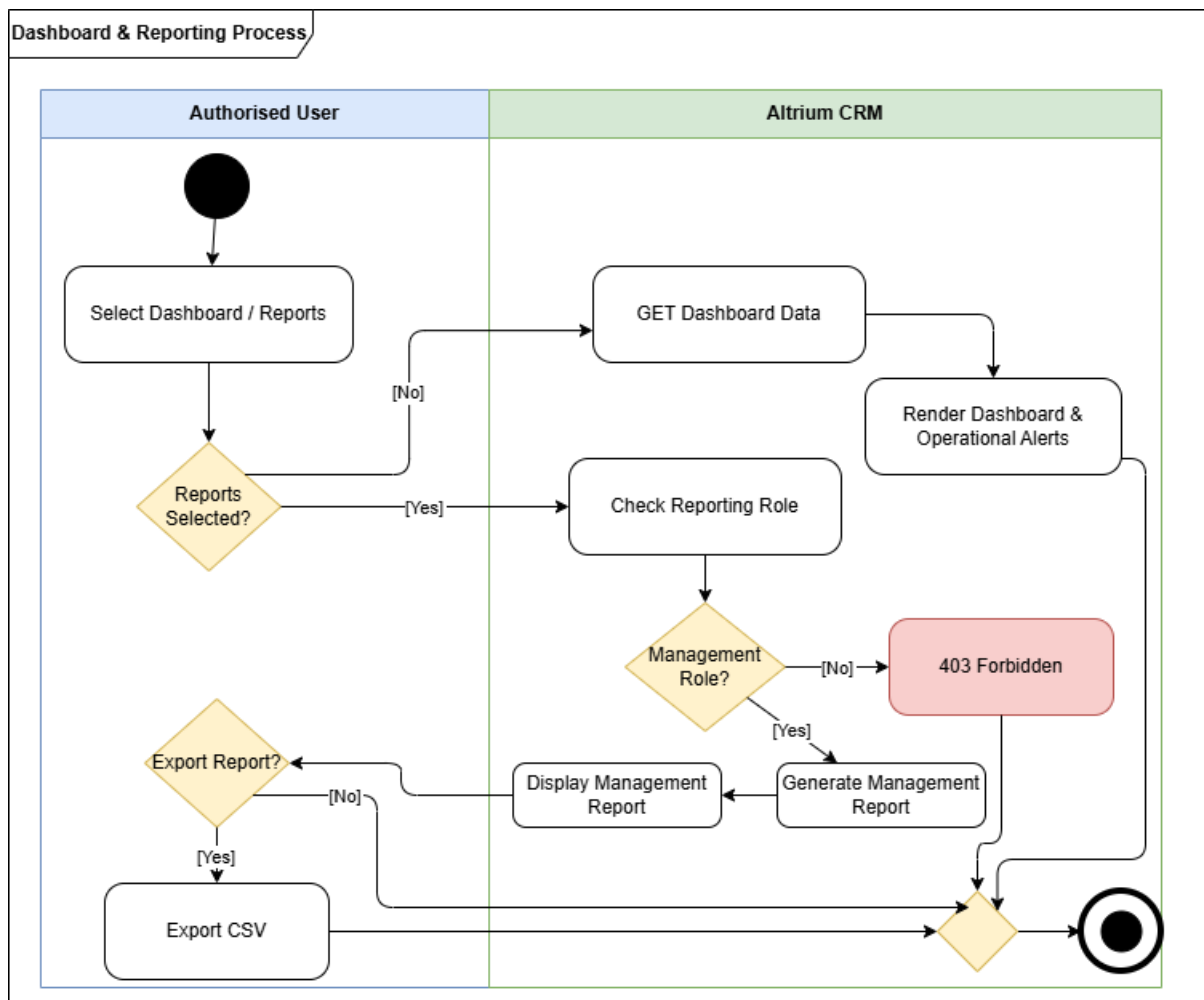


Figure 20: Dashboard & Reporting Process Activity Diagram

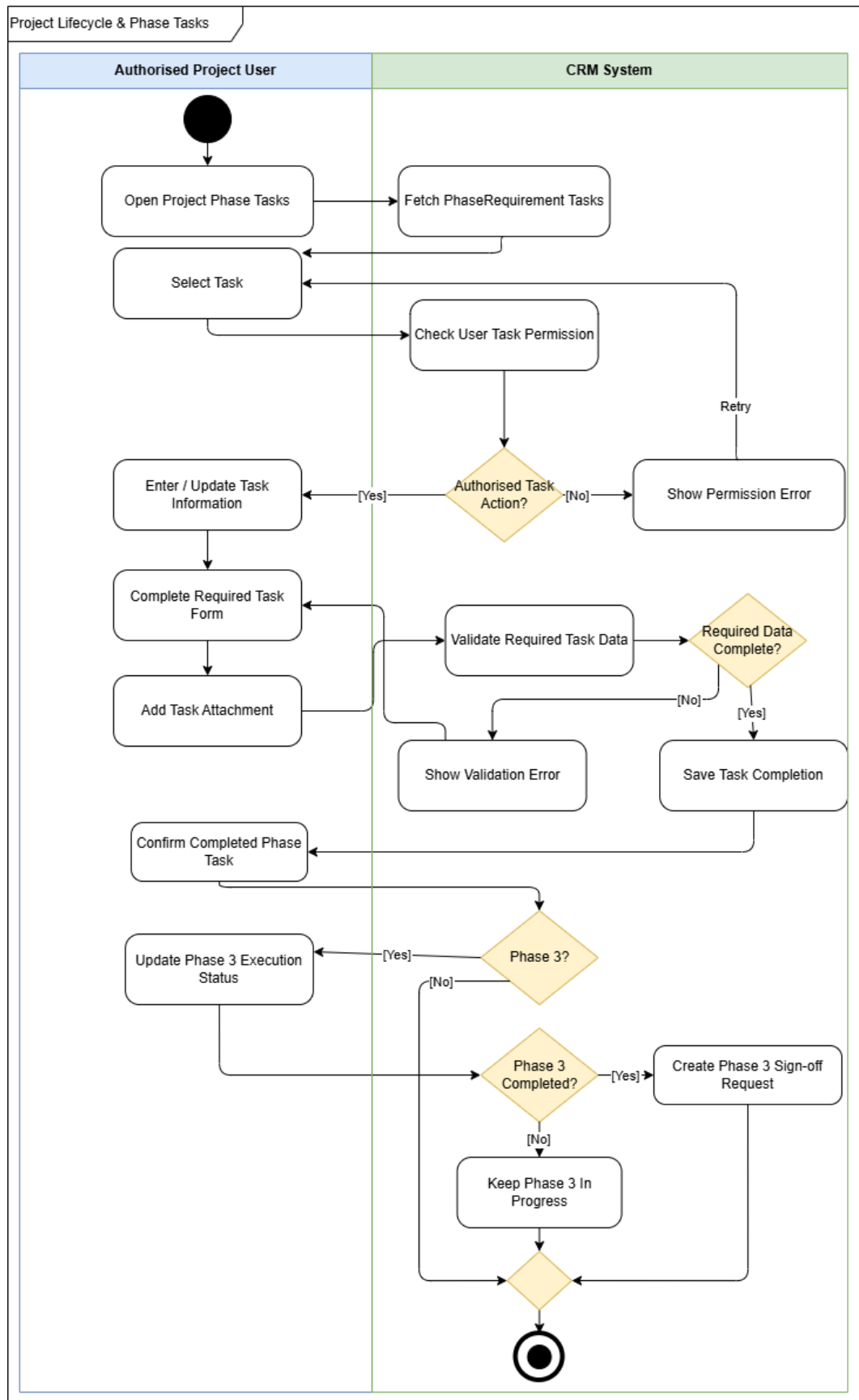


Figure 21: Project Lifecycle & Phase Tasks Activity Diagram

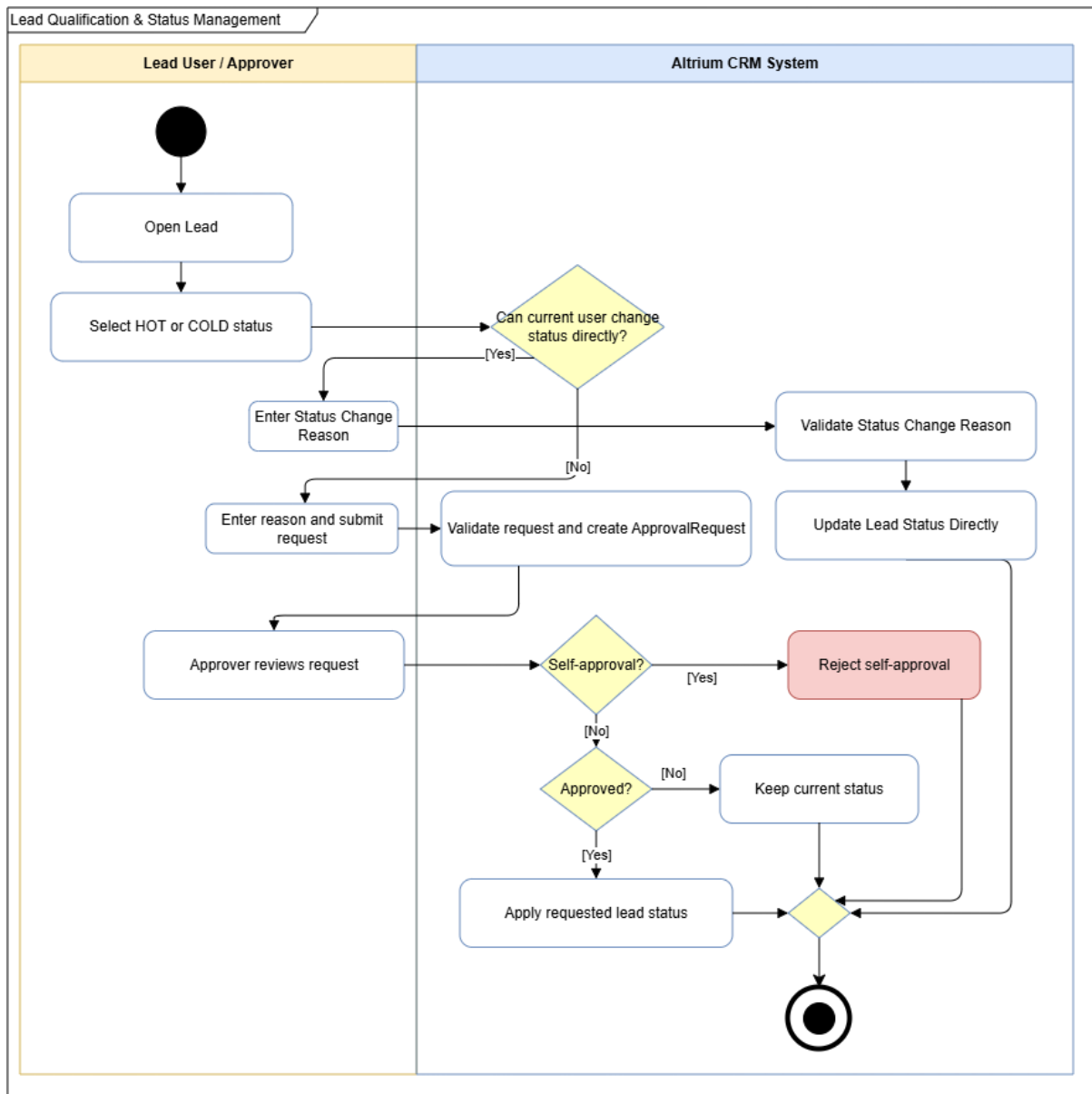


Figure 22: Lead Qualification & Status Management Activity Diagram

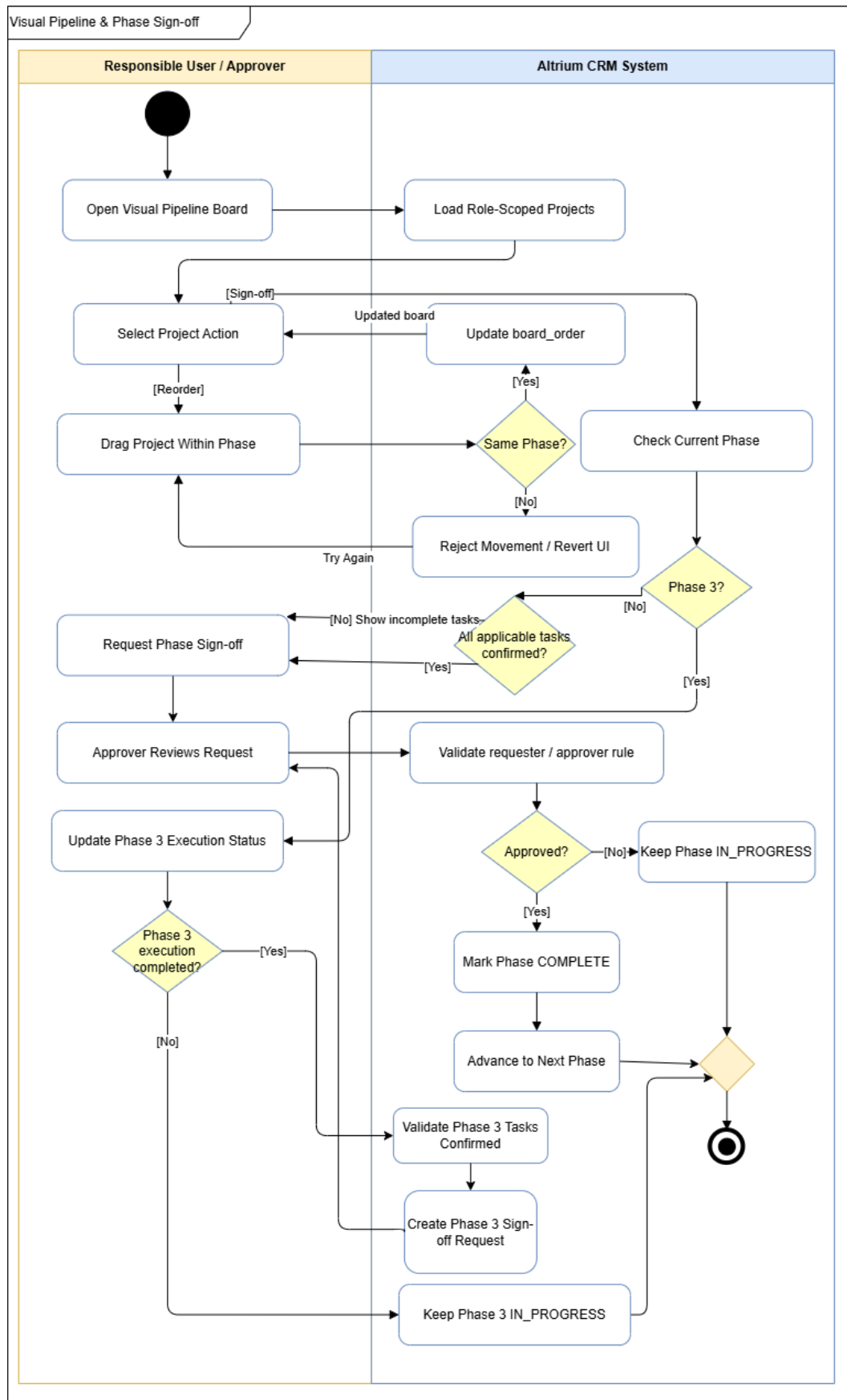


Figure 23: Visual Pipeline & Phase Sign-off Activity Diagram

2.5 Sequence Diagrams

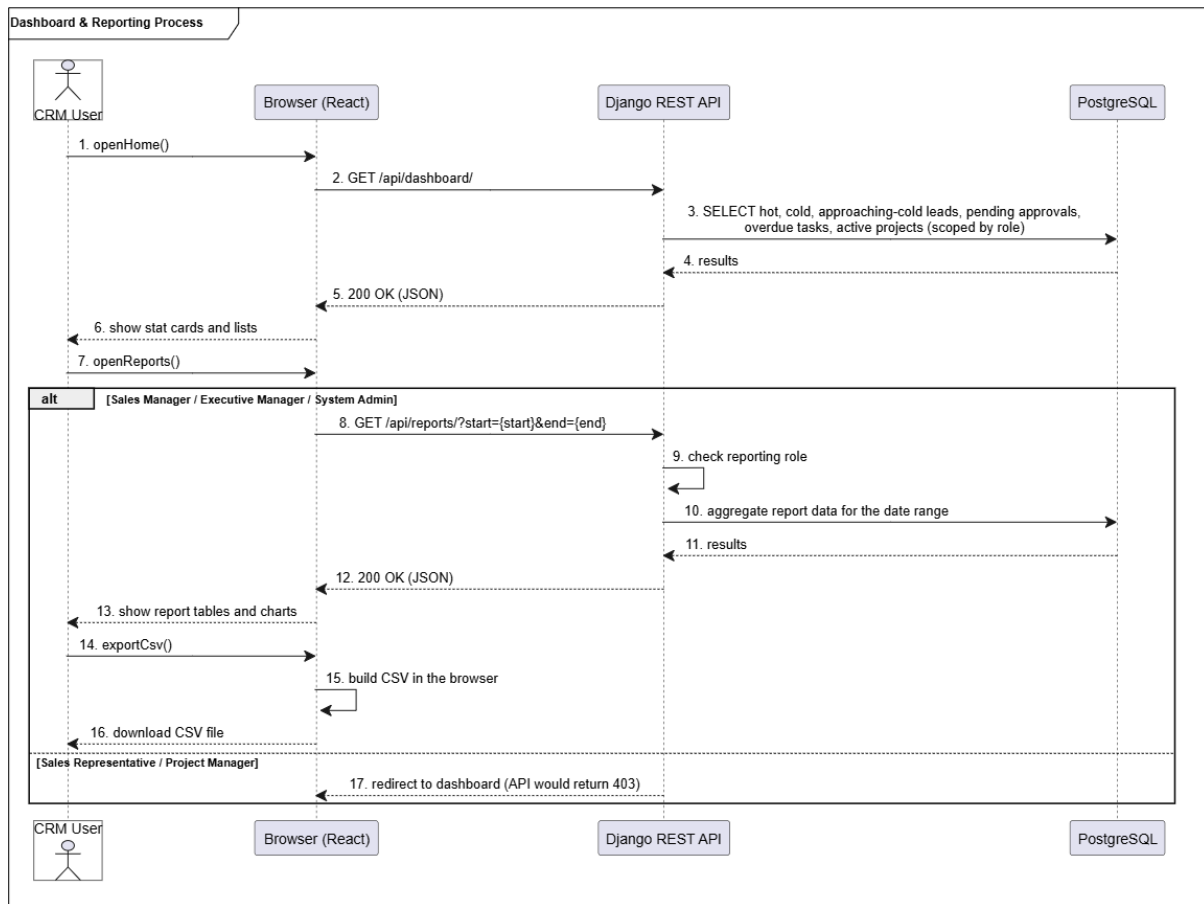


Figure 24: Dashboard & Reporting Process Sequence Diagram

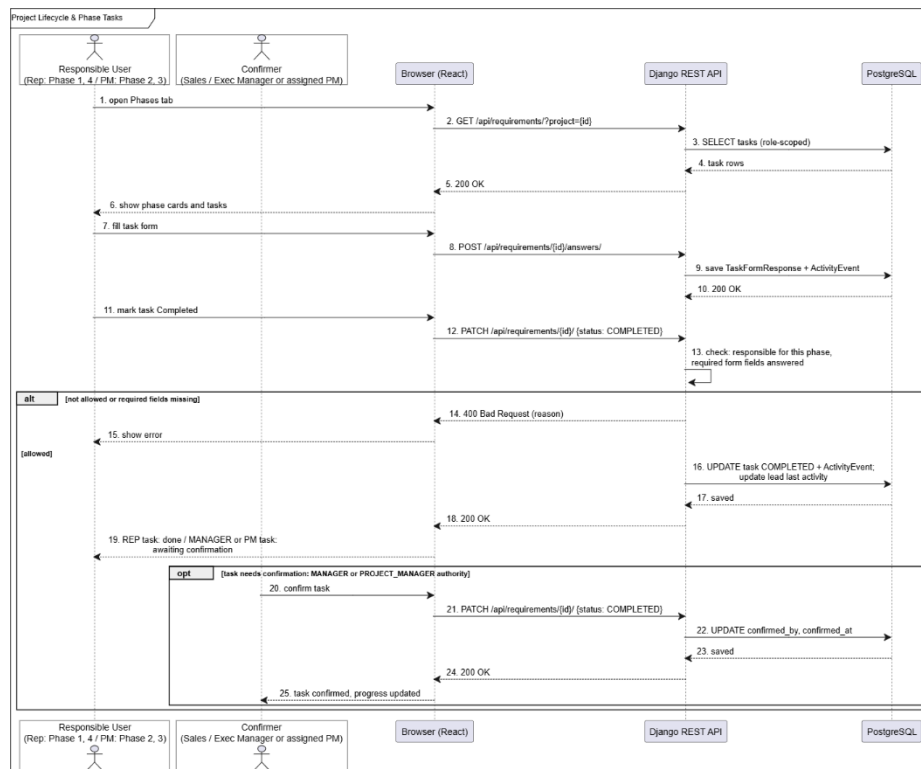


Figure 25: Project Lifecycle & Phase Tasks Sequence Diagram

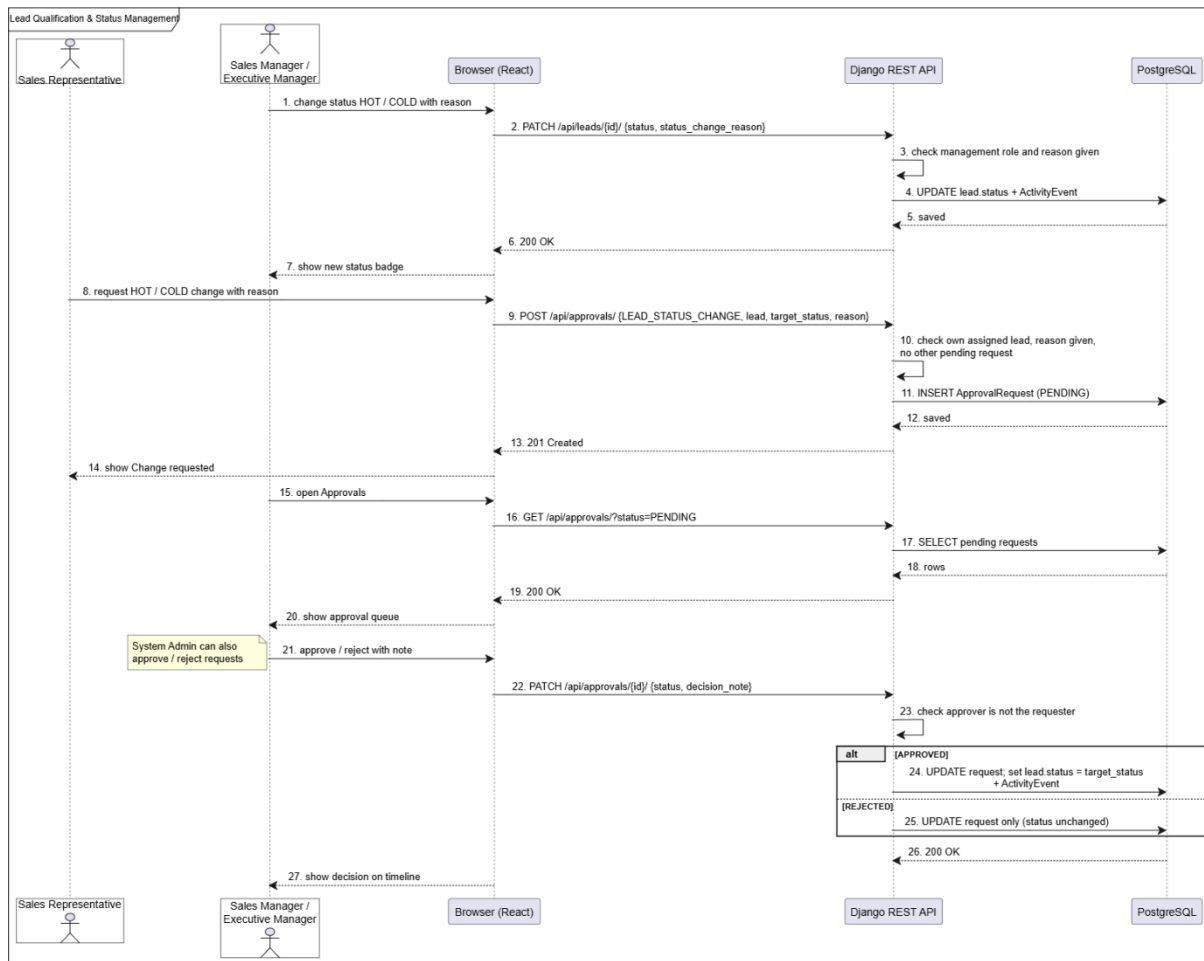


Figure 26: Lead Qualification & Status Management Sequence Diagram

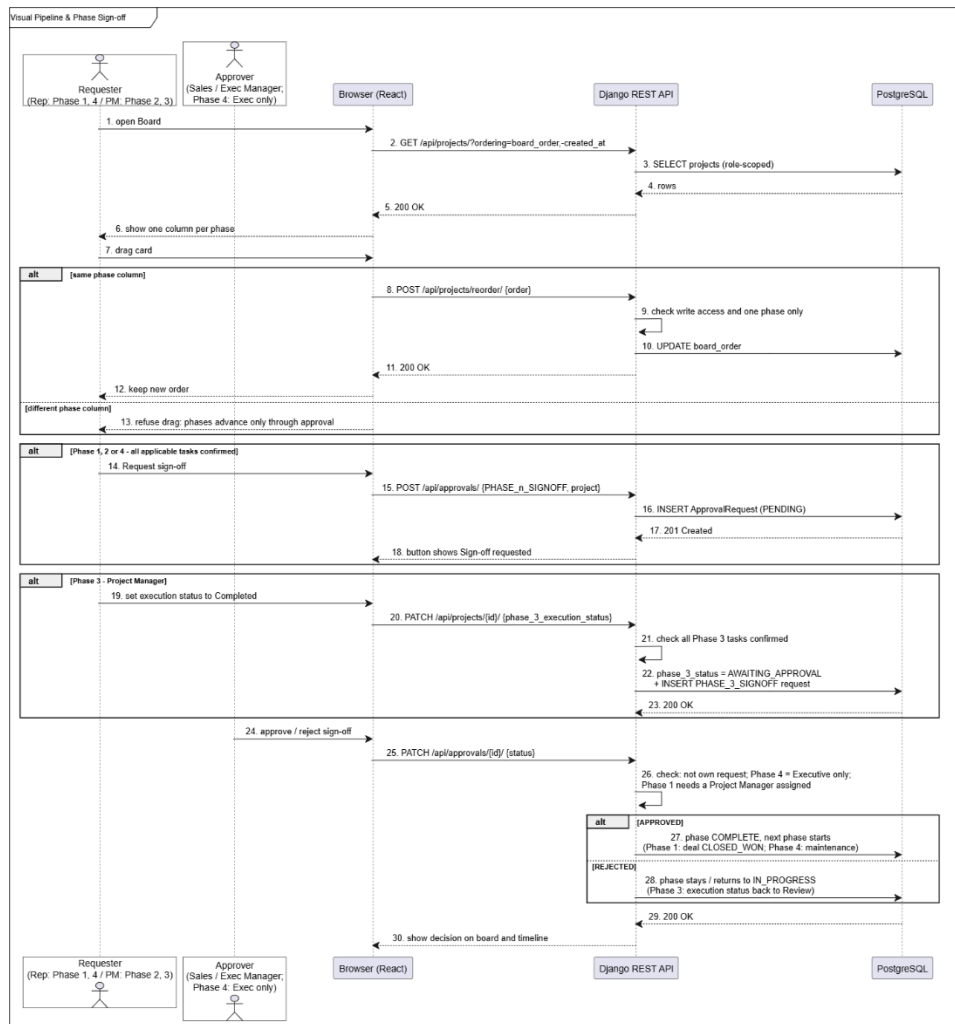


Figure 27: Visual Pipeline & Phase Sign-off Sequence Diagram

2.6 Evidence of testing

2.6.1 Manual Test Cases

Feature #	Feature name	Steps	Expected outcome	Actual outcome	Status
F03b	Lead Status (Sales Rep request)	1. Log in as rep1 and open an assigned lead. 2. Request a change to Hot with a reason.	The status does not change. A pending status-change request is raised for a manager.	The status did not change and a pending status-change request was raised.	Pass
F03c	Lead Status (Manager, direct)	1. Log in as mgr1 and open the lead. 2. Change Status to Hot and give a reason.	The status saves as Hot and the reason appears on the timeline. Saving without a reason is refused.	The status saved as Hot and the reason appeared on the timeline. Saving without a reason was refused.	Pass
F03h	Lead Status (Approve request)	1. As mgr1, open Approvals. 2. Approve rep1's status request.	The lead changes to the requested status.	The lead changed to the requested status after approval.	Pass
F04a	Phase Task (Rep completes own task)	1. As rep1, open the lead and go to Phases. 2. Mark a REP-authority Phase 1 task as Completed.	The task is done straight away and progress moves.	The task turned green immediately and the progress bar moved.	Pass
F04b	Phase Task (Rep completes manager task)	1. As rep1, mark a manager task as Completed. 2. Look at the progress bar.	The task waits for a manager and does not count yet.	The task showed an amber clock saying it was awaiting confirmation, and progress did not move. The rep could not sign it off themselves.	Pass
F04c	Phase Task (Manager confirms)	1. Log in as mgr1 and open the same task. 2. Click Confirm.	The task turns green and progress moves.	It turned green, showed "Confirmed by mgr1", and the progress bar moved up.	Pass
F04d	Phase Task (Not applicable)	1. Open any remaining Phase 1 task. 2. Set it to Not Applicable and save.	The task is greyed out, drops out of the progress count and does not block sign-off.	The task greyed out, dropped out of the progress count and did not block sign-off.	Pass
F04e	Phase Task (Overdue)	1. Open a task and set a committed date in the past. 2. Save and go back to Home.	The task shows as overdue on the lead and on the dashboard.	The task was flagged overdue on the lead and appeared in the dashboard's Overdue Tasks card.	Pass
F04f	Phase Task (Rep blocked on Phase 2)	1. As rep1, open a lead in Phase 2. 2. Try to complete a Phase 2 task.	Refused: only the assigned Project Manager can complete it.	Refused: only the assigned Project Manager can complete the task.	Pass
F04g	Phase Task (Required form field)	1. As pm1, open Budget Proposal.	Step 2 is refused and lists the missing fields. Step 3 completes the task and	Completion was refused and the missing fields were listed. After filling	Pass

		2. Try to complete it with the required fields empty. 3. Fill them in and complete.	the budget appears on the lead.	them in, the task completed and the budget appeared on the lead.	
F04h	Task Attachment	1. Open a task. 2. Upload a file and save.	The file is listed on the task and opens in place.	The file was listed on the task and opened in place.	Pass
F04i	Custom Task	1. As mgr1, click + Add task on a phase. 2. Enter a name and due date and save.	The new task appears in that phase and on the timeline.	The new task appeared in the phase and on the timeline.	Pass
F05a	Sign-off (Request)	1. As rep1, finish the remaining Phase 1 tasks. 2. Click Request sign-off, then refresh.	The request is sent and the button changes to Sign-off requested.	The request was sent and the button changed to "Sign-off requested" and greyed out. It stayed that way after refreshing, so you cannot ask twice by mistake.	Pass
F05b	Sign-off (Duplicate request)	1. With the request still waiting, open the lead in another tab. 2. Try to request Phase 1 sign-off again.	The second request is refused.	Refused with a message saying a request of this type is already waiting. Only one request stayed in the list.	Pass
F05c	Sign-off (Approve Phase 1)	1. As mgr1, assign pm1 as Project Manager. 2. Approve the Phase 1 request.	Phase 1 completes, the deal becomes Closed-Won, and Phase 2 starts with its 5 tasks.	Phase 1 completed, the deal changed to Closed-Won, and Phase 2 started with its 5 tasks.	Pass
F05d	Sign-off (Reject)	1. As pm1, finish Phase 2 and request sign-off. 2. As mgr1, reject it with a note.	Phase 2 stays In Progress and the note shows on the timeline.	Phase 2 stayed In Progress and the manager's note showed on the timeline.	Pass
F05e	Sign-off (Approve your own request)	1. As mgr1, raise a sign-off request. 2. Try to approve it as mgr1.	The person who raised it cannot approve it.	mgr1 was blocked from approving their own request.	Pass
F05f	Sign-off (Executive approves manager)	1. Log in as ex1. 2. Approve the request mgr1 raised above.	The executive can approve a request a manager raised.	ex1 approved it with no trouble, so every sign-off gets a second pair of eyes.	Pass
F05g	Sign-off (No PM assigned)	1. With no Project Manager assigned, approve a Phase 1 request as mgr1.	Refused: a Project Manager must be assigned first.	Approval was refused until a Project Manager was assigned.	Pass
F05h	Sign-off (Phase 3, automatic)	1. As pm1, confirm all Phase 3 tasks.	Phase 3 moves to Awaiting Approval and a sign-off request is raised automatically.	Phase 3 moved to Awaiting Approval and a sign-off request was raised automatically.	Pass

		2. Set execution status to Completed.			
F05i	Sign-off (Phase 4, Executive only)	1. As mgr1, try to approve a Phase 4 request. 2. Repeat as ex1.	mgr1 is refused. ex1 approves, and the project moves to maintenance.	mgr1 was refused. ex1 approved it and the project moved to maintenance.	Pass
F06c	Interaction (Client-facing task)	1. Complete a client-facing task on a Cold lead.	"Last client contact" updates. The lead stays Cold.	"Last client contact" updated and the lead stayed Cold.	Pass
F08a	Dashboard (Sales Rep)	1. Log in as rep1 and open Home.	The rep sees only their own leads.	rep1 saw 2 hot and 1 cold lead, all assigned to them. None of rep2's leads showed up.	Pass
F08b	Dashboard (Sales Manager)	1. Log in as mgr1 and open Home.	The manager sees the whole team's leads, pending approvals and overdue tasks. The overdue count matches F04e.	mgr1 saw the whole team's leads, the pending approvals and the overdue tasks, and the overdue count matched F04e.	Pass
F09a	System Settings (Sales Manager)	1. As mgr1, open Settings. 2. Change the cold lead threshold to 30 days and save.	The setting saves and the Approaching Cold card changes.	Saved as 30 days and the Approaching Cold card went from 1 lead to none, so the number really is a setting and not fixed at 14.	Pass
F09b	System Settings (Sales Rep)	1. Log in as rep1. 2. Try to change the cold lead threshold.	The rep cannot change system settings.	The change was refused. Only manager and admin roles can edit settings.	Pass
F09c	Requirement Templates (Edit)	1. As mgr1, change the due-after days on a Phase 1 task. 2. Open a lead created before the change. 3. Create a new lead and check it.	Old leads keep their dates; only new ones use the new setting.	The old lead kept its original due date of 1 September. Only the new lead picked up the changed number.	Pass
F09d	Requirement Templates (Copy to phase)	1. As mgr1, drag a template into another phase.	A copy appears in that phase with the same form fields. Copying into a phase that already has it is refused.	A copy appeared in the target phase with the same form fields. Copying into a phase that already had it was refused.	Pass
F10a	Project Manager access	1. Log in as pm1. 2. Open a lead on a project they manage, then one they don't manage.	pm1 can work the Phase 2 and 3 tasks on their own project. The other lead is read-only, and pm1 cannot edit lead details or add companies anywhere.	pm1 could work Phase 2 and 3 tasks on their own project. The other lead was read-only, and editing lead details or adding companies was refused.	Pass
F11a	Display Preferences	1. Switch between dark and light mode. 2. Change font size and density in Preferences.	All three settings survive a refresh and a new login.	Dark mode, font size and density all stayed as we set them after refreshing and after logging out and back in.	Pass

		3. Refresh, then log out and back in.			
F12a	Board (Reorder)	1. Open Board. 2. Drag a card within its column. 3. Refresh.	The new order is kept after refresh.	The new order stayed after refreshing.	Pass
F12b	Board (Cross-column drag)	1. Drag a card into another phase column.	Refused, with a message that phases only move through approval.	The drag was refused with a message that phases only move through approval.	Pass
F13a	Reports (Management)	1. As mgr1, open Reports. 2. Change the date range. 3. Click Export CSV.	The report reloads for the new range and a CSV file downloads.	The report reloaded for the new range and a CSV file downloaded.	Pass
F13b	Reports (Rep)	1. As rep1, go to /reports.	The rep is sent back to the dashboard.	rep1 was sent back to the dashboard.	Pass
F14a	Calendar	1. As rep1, open Calendar.	Only tasks on rep1's leads appear, grouped by due date.	Only tasks on rep1's leads appeared, grouped by due date.	Pass
F15a	Notifications	1. As mgr1, log a note tagging a Phase 2 task (#id). 2. Log in as pm1 and open notifications. 3. Click Mark all read.	pm1 sees the notification linking to the task. After marking all read, the unread count is 0.	pm1 saw the notification linking to the task, and the unread count dropped to 0 after Mark all read.	Pass

Table 5: Sprint 2 Manual Test Cases

2.6.2 Automated Test Cases

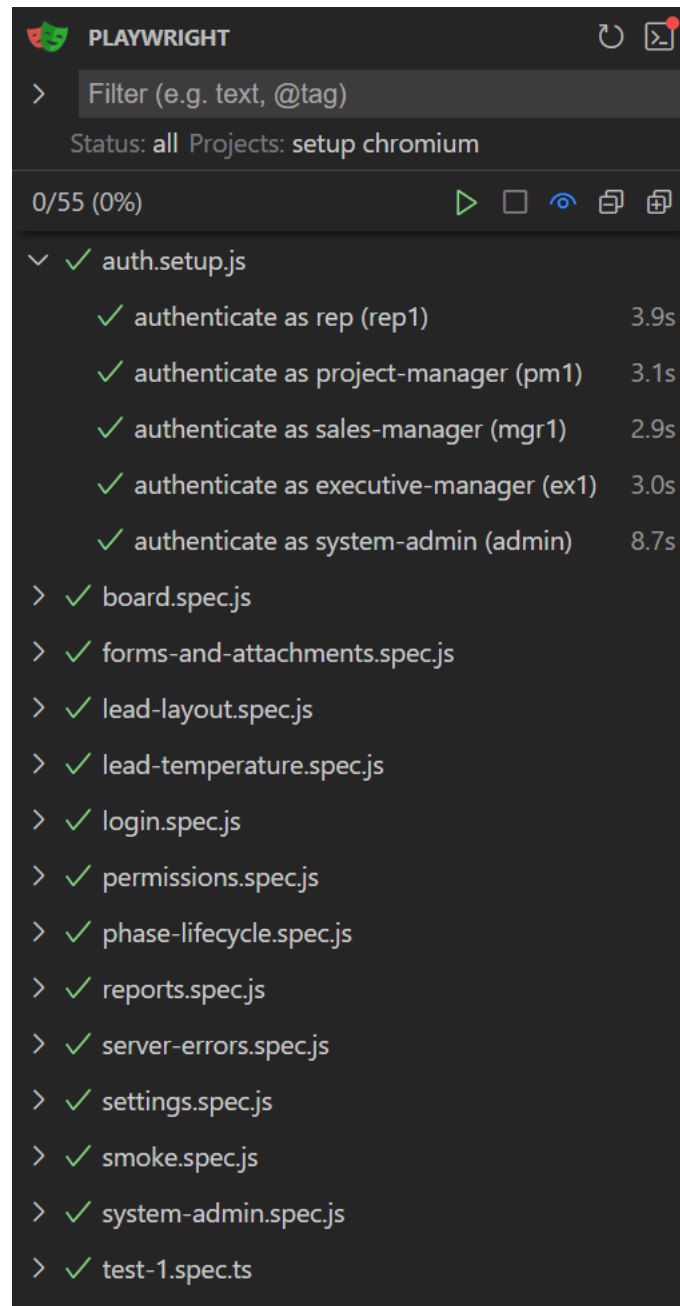


Figure 28: Frontend Automated Testing Results

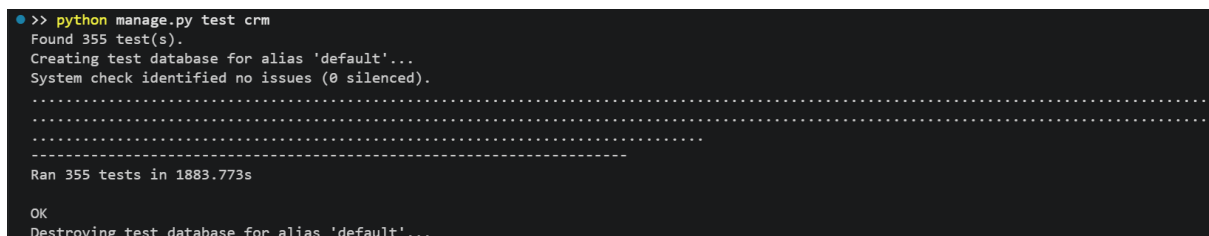


Figure 29: Backend Automated Testing Results

2.7 Sprint Retrospective

2.7.1 What went well in the sprint

- Expanded the CRM into the controlled four-phase project lifecycle.
- Added role-based permissions, task confirmation, approvals, visual board, reporting, calendar, forms, attachments, and settings.
- Improved frontend/backend/database/testing/documentation integration.
- Used automated testing and CI.

2.7.2 What went wrong in the sprint

- Requirements, UML diagrams, and implementation changed during development.
- Some earlier documentation described features that were later removed/replaced.
- Role permissions and phase lifecycle needed refinement.

2.7.3 What we commit to improve

- Keep Product Backlog, Sprint Backlog, User Stories, UML, software, tests, and Definition of Done aligned.
- Review implementation against documentation before submission.
- Involve development, BA, and QA during reviews.

2.7.4 Individual Retrospectives

Member	Roles	Experience	Reflection
Thehan (CB015685)	Developer	• Developed React UI components and implemented frontend functionality for the CRM. • Worked with backend features and role-based UI permissions.	During Sprint 2, I improved my understanding of connecting frontend and backend functionality. I also gained more experience with role-based features and improved my frontend development skills. Compared with Sprint 1, I communicated more with the team to make sure the implemented features matched the project requirements.
	Quality Analyst	Created automated end-to-end tests using Playwright for key CRM features. Performed backend testing with Django and checked permissions and system functions.	Working as a Quality Analyst helped me understand automated testing more clearly. I learned how to create Playwright end-to-end tests and test Django backend functions. I also learned that testing throughout development helps identify problems earlier and improves the reliability of the final system.
Thumula (CB015674)	Business Analyst	Reviewed implemented Sprint 2 features with developers and refined system requirements.	During Sprint 2, I improved my understanding of keeping requirements and UML diagrams connected to the actual implementation. I reviewed the system with developers before finalizing the documentation. This helped me improve my Coordinator skills and understand

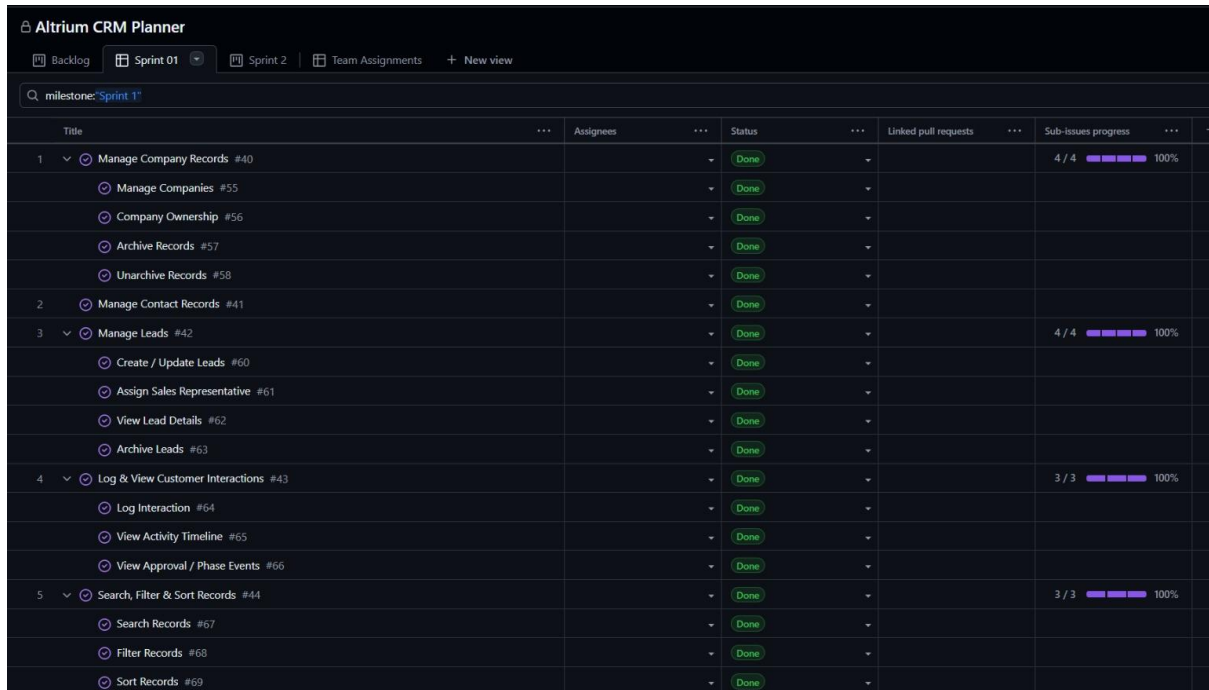
		Created and updated UML diagrams to reflect implemented roles, workflows, and permissions.	the importance of continuous communication between Business Analysis and development.
Sabeeshan (CB011535)	Quality Analyst	Tested completed CRM features against requirements and identified functional issues. Communicated identified problems with developers and verified that corrections worked correctly.	Working as a QA helped me understand how to test a system from the user's point of view. I became more confident in identifying different possible issues and checking whether features met their requirements. I also improved my attention to detail and communication with the development team.
	Scrum Master	Facilitated daily meetings and coordinated team activities during Sprint 2. Followed task progress and communicated pending work and responsibilities to team members.	Working as Scrum Master helped me improve my communication and coordination skills. I became more comfortable with Agile activities because I had previous experience from Sprint 1. I learned that regularly following task progress and communicating with team members is important for keeping the team organized.
Sadev (CB015733)	Scrum Master	Coordinated Sprint 2 activities and followed team progress throughout development. Helped communicate pending tasks and supported the team in completing sprint work.	Working as Scrum Master helped me improve my communication, coordination, and teamwork skills. I learned that regularly checking progress and communicating with team members helps identify pending work early. This experience also increased my confidence in taking responsibility for coordinating Agile activities.
	Developer	Developed and integrated assigned CRM features during Sprint 2. Worked with the team to verify that implemented features matched the planned requirements.	As a Developer, I improved my understanding of implementing features within an Agile team. I learned the importance of communicating with other team members and checking that development work matches the requirements. This experience also helped me improve my technical skills and understand the connection between development, testing, and documentation.

Table 6: Sprint 2 Reflection Table

Appendix

3.1 Project Management SCRUM board

3.1.1 Sprint 1



The screenshot displays the Altrium CRM Planner interface for Sprint 1. The board is organized into columns: Title, Assignees, Status, Linked pull requests, and Sub-issues progress. The tasks are listed in a table format, with each task having a status of 'Done' and a progress bar indicating 100% completion.

Title	Assignees	Status	Linked pull requests	Sub-issues progress
1 Manage Company Records #40		Done		4 / 4 100%
Manage Companies #55		Done		
Company Ownership #56		Done		
Archive Records #57		Done		
Unarchive Records #58		Done		
2 Manage Contact Records #41		Done		
3 Manage Leads #42		Done		4 / 4 100%
Create / Update Leads #60		Done		
Assign Sales Representative #61		Done		
View Lead Details #62		Done		
Archive Leads #63		Done		
4 Log & View Customer Interactions #43		Done		3 / 3 100%
Log Interaction #64		Done		
View Activity Timeline #65		Done		
View Approval / Phase Events #66		Done		
5 Search, Filter & Sort Records #44		Done		3 / 3 100%
Search Records #67		Done		
Filter Records #68		Done		
Sort Records #69		Done		

Figure 30: Sprint 1 Scrum Board

3.1.2 Sprint 2

Altrium CRM Planner									
Backlog Sprint 01 Sprint 2 Team Assignments New view									
Q milestone: Sprint 2									
Title	...	Assignees	...	Status	...	Linked pull requests	...	Sub-issues progress	...
1 Manage Lead Temperature & Status #45				Done				5 / 5 100%	
View HOT/COLD Status #70				Done					
Direct Status Change #71				Done					
Request Status Change #72				Done					
Approve / Reject Status Request #73				Done					
Prevent Self-Approval #79				Done					
2 Manage Pipeline & Phase Sign-offs #46				Done				6 / 6 100%	
View Pipeline Board #74				Done					
Reorder Projects #75				Done					
Request Phase Sign-off #76				Done					
Approve / Reject Sign-off #77				Done					
Prevent Self-Approval #78				Done					
Approve Phase 4 Sign-off #80				Done					
3 Manage Project Phases & Tasks #47				Done				9 / 9 100%	
Assign Project Manager #81				Done					
Manage Phase Tasks #82				Done					
Complete Phase Tasks #83				Done					
Confirm Completed Phase Tasks #84				Done					
Manage Task Forms #85				Done					
Add Task Attachments #86				Done					
Manage Task Due Dates #87				Done					
Manage Phase 3 Execution Status #88				Done					
Archive Projects #89				Done					
4 View Dashboards & Operational Alerts #48				Done				3 / 3 100%	
View Lead Status Dashboard #90				Done					
View Pending Approvals #91				Done					
View Overdue Tasks #92				Done					
5 Generate Management Reports #49				Done					
6 Manage Requirement Templates #50				Done					
7 Manage System Settings #51				Done					
8 Manage Personal Preferences #52				Done					
9 View Task Calendar #53				Done					
10 Manage Mentions & Notifications #54				Done					

Figure 31: Sprint 2 Scrum Board

3.2 Meeting Minutes of Daily Standup Meetings

Meeting 38 — Sprint 2 Kickoff

Date: 18 August 2026 Time: 02:00 P.M.

Attendees

- Thehan Andaramana, Thumula Rumes, Sadev Ravishan, Sivakumar Sabeeshan

Discussion

- Reviewed the Sprint 1 mockup test outcome and confirmed the framework/workflow choices going into real development.
- Broke down Sprint 2 scope into four core areas: Lead Qualification & Status Management, Visual Pipeline & Phase Sign-offs, Project Lifecycle & Phase Tasks, and Dashboard & Reports.
- Assigned rough ownership across the team for backend, frontend and testing work.

Decisions

- Proceed with the four-area Sprint 2 breakdown.
- Architecture confirmed: React frontend, Django REST API backend, PostgreSQL database.

Completed

- Sprint 2 scope and task board set up.

Figure 32 : 38th Meeting

Meeting 51 — Project Lifecycle & Phase Tasks Design

Date: 31 August 2026 Time: 10:00 A.M.

Attendees

- Thehan Andaramana, Thumula Rumes, Sadev Ravishan, Sivakumar Sabeeshan

Discussion

- Designed the phase task model: due dates, custom task forms, attachments.
- Discussed task confirmation — manager-authority tasks confirmed by management, Project-Manager-authority tasks confirmed by the assigned Project Manager.
- Reviewed Phase 1/4 (Sales Representative tasks) vs Phase 2/3 (Project Manager tasks) split.

Decisions

- Task confirmation rules finalized per role/phase.

Completed

- Project Lifecycle & Phase Tasks design finalized.

Figure 33: 51st Meeting

3.3 Risk Register

ID	Risk	Before Mitigation		Before Risk Score	Mitigating Actions	After Mitigation		After Risk Score	Final Risk Level
		Likelihood (1–5)	Severity (1–5)			Likelihood (1–5)	Severity (1–5)		
1	Requirements and UML diagrams become inconsistent with the implemented system.	4	4	16	Review requirements and UML with developers throughout the sprint and update documentation when changes occur.	2	3	6	Low
2	Frontend and backend integration problems delay feature completion.	4	4	16	Test API integration during development and review frontend-backend functionality before marking tasks as Done.	2	3	6	Low
3	New requirements or client feedback increase the project scope during development.	4	4	16	Record changes in the Product Backlog and prioritise them; protect the current sprint from unnecessary scope changes.	2	3	6	Low
4	Testing is delayed until the end of the sprint, causing defects to be found late.	4	4	16	Involve QA during planning and prepare test cases alongside development instead of	2	2	4	Low

					testing only at the end.				
5	Incorrect role permissions allow users to access or modify functions they should not use.	3	5	15	Maintain a role-permission matrix and test permissions for each user role in both frontend and backend.	1	4	4	Low
6	Poor communication between team members causes tasks and documentation to become misaligned.	4	4	16	Use regular stand-ups, keep the GitHub board updated, and conduct cross-role reviews between BA, developers and QA.	2	3	6	Low
7	Limited sprint time results in incomplete or low-priority features.	4	4	16	Prioritise Must/Should features, divide work into smaller tasks, and monitor progress regularly using the Scrum board.	2	3	6	Low
8	Development environment or PostgreSQL/Docker problems interrupt development.	3	4	12	Maintain setup instructions, use a consistent Docker configuration, and check the development environment before sprint work.	1	3	3	Low
9	Incomplete or unclear user stories cause developers to implement the wrong functionality.	3	4	12	Review user stories with the team before development and confirm that the	2	3	6	Low

					implemented feature matches the intended requirement.				
10	Git merge conflicts or incorrect changes affect completed development work.	3	3	9	Use separate branches, pull the latest changes before development, and review changes before merging.	1	2	2	Low

Table 7 : Meeting Register